

Learning, Search and Evidence in a Pokémon Showdown VGC Agent

Architecture, Data, Training on CPU and Apple MPS, Decision-Time Search,
and the Mathematics Behind Each Claim

`pokemon-showdown-player`

Technical report grounded in the repository implementation

9 October 2026 (second edition; first edition 7 October 2026)

Public source snapshot: `0c00ec2`; simulator pin `c046106`; feature schema 1

Abstract

This report describes a complete system for playing the Champions VGC 2026 Regulation M-C doubles format on Pokémon Showdown, and it audits the mathematics behind each component. Turn decisions are made by a *determinized simultaneous-move search* run in the official simulator, over opponent sets sampled from ladder usage statistics. A compact action-conditioned actor-critic chooses team previews and generates all training data. A Playwright browser transport clicks the decisions into the real client. A supervisor schedules three bounded lanes on one Apple M5 laptop: CPU self-play collection, PPO learning on the Metal (MPS) GPU, and paired CPU evaluation. A candidate is promoted only after an exact paired sign test and a set of integrity checks.

We give the exact network, the strategic prior and its temperature, the per-decision record kept for every match, the PPO loss and its KL stop, the resource and promotion rules, and the search algorithm. Thirty-one propositions, theorems and lemmas state their assumptions and are proved. Eight worked examples recompute the implemented numbers. A new evidence chapter reports the paired offline result (search 30/45 against the learned policy’s 19/45, McNemar $p = 0.0064$), the live record before and after search, and a negative-results ledger of more than thirty rejected candidates. This edition also corrects the first: it records stale claims, a circular proposition and an unstated temperature, and it documents implementation defects found during review.

Keywords: Pokémon Showdown, VGC doubles, determinization, regret matching, PPO, action masking, Apple MPS, paired sign test, Elo.

Contents

1	Introduction and scope	4
2	System overview	5
3	The battle as a partially observed stochastic game	8
4	Candidate actions and doubles constraints	10
5	Features, profiles and the strategic prior	12
6	The action-conditioned actor–critic	15
7	The data recorded for every match	17
8	Learning from outcomes: policy gradients, GAE and PPO	20
9	The training pipeline on CPU and Apple MPS	24
10	Promotion: a paired test and what it can certify	27
11	Decision-time search	30
12	One live turn: browser transport, guards and viewer	35
13	Opponent-move scouting and team selection	37
14	Evidence	39
15	Limitations and research directions	43
A	Notation	43
B	Corrections to the first edition	44
C	Implementation defects found during this review	45
D	Source-to-claim map	46
E	Reproduction and build	46

List of Figures

1	Deployed responsibilities on 9 October 2026. Orange boxes are the decision-time search. The learned actor chooses only the team preview live, but it generates all training data in self-play. The dashed promotion path replaces the actor between games, after a candidate has passed the paired gate and integrity checks. The dashed MCP path supports agent-supervised play and recording; it is not used by the browser runner.	6
---	---	---

2	Chronology of the public commits from 153e3db to 0c00ec2 (7–9 October 2026). The results are recomputed in Section 14.	7
3	The deployed actor–critic. The final actor layer is initialized to zero, so a fresh actor equals $\text{softmax}(q_0/\tau)$. The optional <code>matchup-v1</code> variant adds a bilinear term $u^\top(e_a \odot Me_s)$ to the logit.	15
4	The recorder-3 episode. Learning uses the orange arrays; auditing and replay use the snapshot. For the last 20 ladder games the median document was 1.30 MB (range 0.73–2.06 MB). In a 7-step game, feature arrays took 750 KB and snapshots 440 KB of 1.20 MB.	18
5	Data lifecycle and the storage budget. Above 90% of the budget, collection, learning and evaluation defer. Above 85%, finished candidates’ input snapshots are compacted down to 75%. Collecting checkpoints are kept once per digest, so every recorded likelihood can be recomputed.	19
6	GAE on a four-decision episode with a terminal win. Only the last residual contains the reward. Each earlier advantage adds its own residual to the next advantage, discounted by $\gamma\lambda$	21
7	The lifecycle of experience. Orange boxes are gates that can stop the flow. Steps 1 and 7 run on the CPU, step 5 on the MPS GPU. Promotion (step 10) is called by the player between games, never by a worker. Episodes are marked consumed when the candidate is saved, before it is evaluated, so “trained” means <i>used for a candidate</i> , not <i>deployed</i>	24
8	Schematic timeline of the lanes; durations are not to scale. The collector and evaluator share one pool of simulator slots. Within the default budget of four, the evaluator gets half (calibrated locally to three) and the collector the rest. The learner owns the GPU. The live runner is outside the pipeline and only ever consumes a staged candidate at a game boundary.	25
9	The paired design on twelve illustrative seeds. Only the five discordant seeds bear on the comparison. Here $g = 4$, $\ell = 1$ and $p = \mathbb{P}(\text{Bin}(5, \frac{1}{2}) \geq 4) = 0.1875$, so this run would fail.	28
10	The search agent. Each world fixes complete hypothetical opposing sets and copies all public state. Screening keeps at most 24 of our joint actions and 20 of the opponent’s. The payoff matrix of a world therefore needs at most 480 one-turn simulations, plus up to $4 \mathcal{B} + 4 \mathcal{A} $ for screening. Team preview is delegated to the learned actor. . . .	30
11	One live turn under search. Step 8 is the freshness check after the decision. Step 9 installs a capture-phase guard that cancels any click once the server’s rqid has changed. Preview turns replace steps 4–6 with a greedy actor decision. In recorded mode, steps 4–6 are <code>Brain.decide</code>	35
12	Paired evaluations. Filled markers passed their gate and hollow markers failed. The orange diamond is the search agent, compared with the learned policy on the same 45 seeds. Bars are approximate 95% intervals for a paired difference, $\pm 1.96\sqrt{[(g+\ell)/n - ((g-\ell)/n)^2]/n}$. The <code>opening-v1</code> panel did not report its discordant counts. * Met $p \leq 0.05$ but missed the then-required 10-point minimum by one win. † One pass among 129 gates; see Proposition 10.2.	40
13	Live rating under the search agent on 9 October 2026. Gaps are games whose terminal log lacked our rating line. The climb from 1182 to 1464 took place in the symmetric- K region, where Proposition 14.1 predicts no drift for a player of stable strength.	42

1 Introduction and scope

The practical question behind this repository is simple to state. Can one laptop, with no human in the turn loop, play a current VGC format on the public Showdown ladder, improve from its own experience, and prove each improvement before relying on it? Answering it takes several kinds of component, each with its own evidence and failure modes. A transport has to deliver legal choices on time. A decision procedure has to choose them. A data recorder has to preserve exactly what was known when each choice was made. A learner has to turn those records into candidate models. A statistical gate has to decide which candidates are better. This report explains each component, gives the mathematics it relies on, and states what has been measured.

1.1 What changed since the first edition

The first edition (7 October 2026) described a persistent MCP harness in which the OpenClaw agent called tools and a local actor–critic chose every battle decision. Thirty-three public commits later the deployed system is different in five ways that matter mathematically:

1. **Turn decisions come from search.** A determinized one-turn simultaneous-move search over the official simulator chooses live turn actions (Section 11). The neural actor now chooses only team previews, and it does so greedily.
2. **The neural policy has a temperature and a strategic prior.** Logits are divided by $\tau = 0.25$. The additive prior is no longer the legacy power heuristic: it is a squashed one-turn scenario evaluation (Sections 5 and 6).
3. **Promotion is a hypothesis test.** Candidates must pass an exact one-sided paired sign test at $p \leq 0.05$ as well as a win margin. The worker only stages a candidate; promotion happens later at a matchmaking boundary (Section 10).
4. **Training is a three-lane pipeline.** CPU collection, PPO on the Apple MPS GPU and CPU evaluation run as separate bounded processes. Each trajectory is checked against its collecting likelihood, and PPO stops early on a KL criterion (Sections 8 and 9).
5. **Live play is browser-driven.** Playwright clicks the official client, guarded by the request identifier. OpenClaw and MCP remain available for agent-supervised play but are no longer in the turn loop (Section 12).

Appendix B lists every first-edition statement that this edition corrects, including one circular proposition and several claims made stale by later commits.

1.2 Three kinds of statement

The report keeps three kinds of statement apart:

1. **Implemented behaviour:** facts traceable to source functions, such as tensor dimensions, constants and gate predicates. These are followed by an *Implementation* line naming the files.
2. **Conditional mathematical results:** propositions proved under explicit assumptions, followed by an *Interpretation* or *Caveat* that says how far the result transfers to the running system.
3. **Measurements:** results of experiments run by the project, cited to the repository’s validation documents or, where marked, to the local run ledger. This paper ran no new games, training or promotion, apart from read-only recomputation of the statistics it reports.

1.3 Format and pinned mechanics

The main target is `gen9championsvgc2026regmc`: doubles, bring six and pick four, open team sheets on request, one Mega Evolution per battle, no Terastallization, Item and Species Clauses, and Champions stat points instead of EVs and IVs [31]. The local simulator dependency is pinned to official Showdown commit `c046106cbe075931b1ff8d8b800ff5be47a85f96` [21]. A format identifier beginning with `gen9` does not determine these mechanics. Format, simulator revision, feature profile, team fingerprint and collecting checkpoint are part of every scientific statement in this report.

1.4 Summary of measured results

Question	Measured answer (details in Section 14)
Does search beat the learned policy offline?	Same 45 seeds, sides and real opponent teams: search 30/45 vs. policy 19/45; 14 gains, 3 losses; exact McNemar $p = 0.0064$. Clustering by opponent team: $p = 0.0037$.
Did the learned policy improve over campaigns?	Live record 348–499 (41.1%) over 847 games across three actor generations. One strategic prior passed an independent 160-game final (125 vs. 106, $p = 0.0072$). One PPO update passed the 60-game gate out of 129 attempts.
What happened live after search?	Preliminary and observational: 41 games, 27–14. Rating 1182 → 1464. Elo-expected 18.4 wins in 37 rated games; observed 23 (exact $p = 0.086$).
What failed?	More than thirty candidates: PPO variants, preview models, a matchup architecture that passed development and then tied its final, and an offline hyperparameter matrix with no significant arm after Holm correction.

Table 1: Headline evidence. Ladder results depend on the opponent pool and on rating-based matchmaking; Section 14.3 explains why a raw win rate is not a stable strength measure.

1.5 How to read this report

Section 2 gives the system map and project timeline. Sections 3–6 formalize the game, the candidate actions, the features and the neural policy. Section 7 describes the record kept for every match. Sections 8–10 cover learning, the CPU/MPS pipeline and promotion statistics. Section 11 analyses the search agent and Section 12 the live turn. Section 13 covers opponent-move scouting and team selection. Section 14 collects all evidence and Section 15 its limits. The appendices provide a glossary, the corrections to the first edition, defects found during review, a source map and reproduction notes.

2 System overview

2.1 Components and responsibilities

Figure 1 shows the deployed system. It has three columns. The *play* column talks to Showdown. The *decide* column chooses actions. The *learn* column turns recorded games into candidate models and decides whether to deploy them. The arrows that cross columns are narrow on purpose. A decision procedure receives a context built from public information and the player’s own request,

and returns one complete choice string. A learner receives completed, validated records and returns a checkpoint file. Deployment is one atomic file replacement made at a matchmaking boundary.

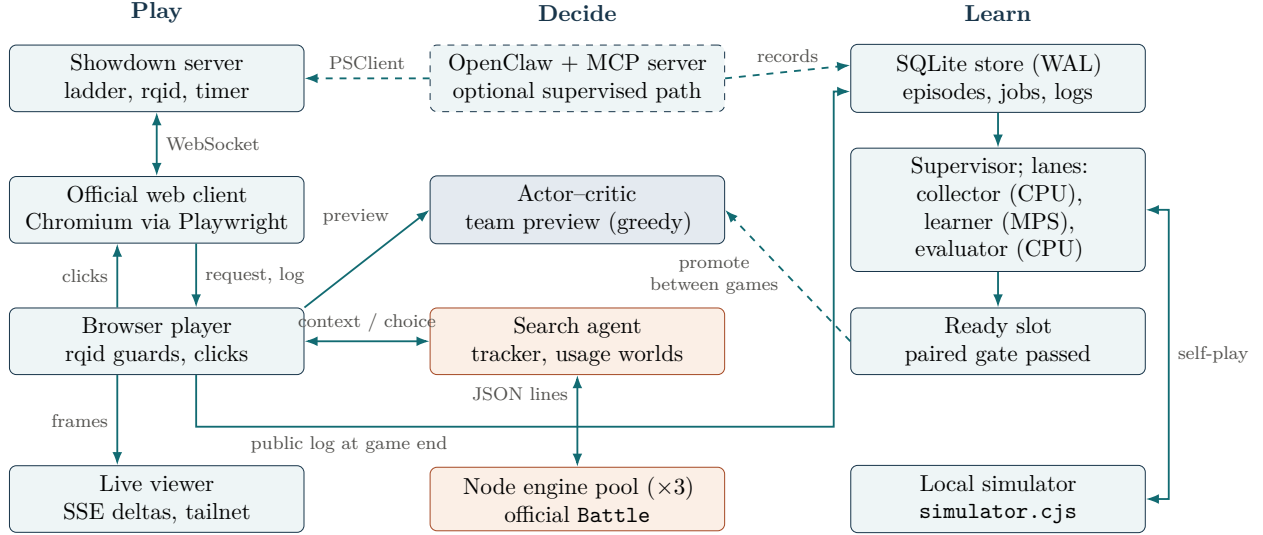


Figure 1: Deployed responsibilities on 9 October 2026. Orange boxes are the decision-time search. The learned actor chooses only the team preview live, but it generates all training data in self-play. The dashed promotion path replaces the actor between games, after a candidate has passed the paired gate and integrity checks. The dashed MCP path supports agent-supervised play and recording; it is not used by the browser runner.

2.2 Processes, devices and cadence

2.3 Project timeline

Figure 2 places the changes analysed in this report in time. The dates explain an important feature of the evidence. The 847-game live record spans three actor generations and two transports, so it is not a measurement of any single policy.

2.4 Two transports, one decision contract

The repository keeps two ways to reach Showdown. Both obey the same contract: a decision is one complete choice string for one actionable request, and it is taken from the request-visible candidate list $\mathcal{A}(r)$ of Section 4.

- **MCP harness.** `ps_mcp_server.py` registers 41 tools. The groups are connection, teams, play, model, learning and research. A single `PSClient` WebSocket logs in with a challenge-string assertion and sends `ROOMID|/choose CHOICE` [15, 16, 20]. OpenClaw’s native registry exposes the tools as `pokemonshowdown__ps_*`. A retained session keeps the server process across turns; a detached one-shot run retires it [17]. This path still records decisions and can resume one pending episode after a disconnect.
- **Browser runner.** `scripts/browser_play.py` drives the official client with Playwright. It reads `app.rooms[id]` and clicks visible controls. Decisions come from local inference or search; no language model is in the turn loop (Section 12).

Process	Device	Role and cadence
Browser runner	CPU	One account lock. Plays ladder games through the official client, with a 30-second gap between games. Calls promotion before each matchmaking search.
Search engines	CPU (3 Node)	Long-lived simulator processes. Each turn they receive 2 of the 6 sampled worlds; median decision 2.3 s.
Supervisor	CPU	launchd KeepAlive . Polls every 15 s, samples host resources, and runs at most one child per lane.
Collector lane	CPU	Self-play in spawned one-thread processes, each driving a Node simulator. Writes complete sampled episodes.
Learner lane	MPS GPU	PPO on validated episodes; at most 12% of recommended Metal memory; falls back to CPU on device errors. Then a scout update.
Evaluator lane	CPU	Paired incumbent and candidate panels with frozen inputs; the exact sign-test gate; staging.
Live viewer	CPU	Publishes filtered public frames every 0.2 s over SSE.
Cold archiver	CPU	Every 900 s, gzip-archives completed inference inputs with SHA-256 verification.

Table 2: Long-running processes on the always-on Apple M5 host (10 cores, 16 GB unified memory). The lanes are described in Section 9.

- 7 Oct • First edition; MCP harness; 100-game campaign (33–67)
- 7 Oct • Paired sign-test gate; turn temperature 0.25 promoted (95→129 of 200, $p \approx 10^{-6}$)
- 7 Oct • Recorder snapshots; feature profiles; colored-HP fix (new profiles only)
- 8 Oct • Integrity pass: likelihood check, recorder 3, 64-bit seeds, **ladder-v1** curriculum
- 8 Oct • Three-lane CPU/MPS pipeline; source-generation reloads
- 8 Oct • 375 games (146–229); opening, Mega and matchup candidates rejected
- 8 Oct • **strategic-v1** prior promoted (final 125 vs. 106 of 160)
- 9 Oct • **strategic-v2..v4** rejected; v4 fails a fresh confirmation
- 9 Oct • Old campaign stopped at 847 games (348–499); browser transport
- 9 Oct • Search agent deployed (offline 30 vs. 19 of 45)
- 9 Oct • PPO update passes the 60-game gate (38→46); 150 GB storage budget
- 9 Oct • Offline PPO matrix: no arm significant after Holm correction

Figure 2: Chronology of the public commits from 153e3db to 0c00ec2 (7–9 October 2026). The results are recomputed in Section 14.

The two paths have different recording rules. MCP play can record sampled PPO data. A browser game played with `-search` is never a PPO sample, because its actions do not come from the actor’s distribution. It contributes only its public log to the scout corpus (Section 13).

Implementation: `ps_mcp_server.py`, `ps_client.py`, `harness.py`, `ml/live.py`, `ml/browser.py`, `scripts/browser_play.py`, `scripts/browser_service.py`, `scripts/learning_service.py`.

3 The battle as a partially observed stochastic game

3.1 State, actions and observations

Let s_t be the complete battle state at a decision boundary. It includes both teams’ hidden sets, current HP, statuses, field conditions, pending mechanics and the simulator’s random-number state. Player 1 chooses a complete action a_t and player 2 chooses b_t simultaneously. Chance ξ_t affects the transition:

$$s_{t+1} \sim P(\cdot \mid s_t, a_t, b_t), \quad o_{t+1}^i \sim O_i(\cdot \mid s_{t+1}), \quad i \in \{1, 2\}. \quad (1)$$

A real trace also contains team preview, forced replacements and waiting requests. In the learner, the index t counts *recorded actionable requests*, not battle turns. Holding the opponent’s strategy fixed reduces the game to a single-agent partially observed control process. If the opponent’s behaviour depends on its own history, that history must be part of the latent state.

Write h_t for the information available to our player before decision t : the public protocol prefix, our own private requests and our past actions. An exact Bayesian belief is $\beta_t(s) = \mathbb{P}(s_t = s \mid h_t)$. For a fixed opponent with joint kernel $\bar{P}$ and observation kernel O , it updates by

$$\beta_{t+1}(s') = \frac{O(o_{t+1} \mid s') \sum_s \bar{P}(s' \mid s, a_t) \beta_t(s)}{\sum_u O(o_{t+1} \mid u) \sum_s \bar{P}(u \mid s, a_t) \beta_t(s)}, \quad (2)$$

whenever the denominator is positive [9, 28]. Nothing in the repository computes Equation (2). The neural policy compresses h_t into a feature vector $x_t = \phi(h_t)$ (Section 5). The search agent replaces the belief with a *sampling distribution* over opposing sets, built from ladder usage statistics and conditioned on what has been revealed (Section 11). Both are approximations to β_t , and neither is calibrated.

3.2 What the format reveals

Champions M-C makes some hidden information public. Team preview reveals all six opposing species. When open team sheets are exchanged, the opposing species, items, abilities, moves and natures become known. Stat-point allocations stay hidden, and IVs are unused in this format. The audit in `docs/match-data-collection.md` found this pattern directly. Over 50 audited games, our own nature and stat points were stored in all 50. The opponent’s nature was disclosed in 2, and its stat points in none. Open sheets are therefore informative but not complete. Only the damage-relevant stat spread remains latent once sheets are shown, but that spread alone moves damage rolls across KO thresholds.

Interpretation. An opposing Pokémon at 50% HP is an observation. Its unrevealed item is a latent variable. A predicted Protect is a learned forecast. A sampled stat spread is one hypothesis drawn from a population model. None of these is a recovered hidden state.

3.3 Non-anticipation

Both decision procedures read only the request and the public prefix. This can be stated precisely.

Proposition 3.1 (Conditional non-anticipation). *Fix the auxiliary inputs: the encoder and its feature profile, the cached dex, the checkpoint (including its embedded `strategy_knowledge`), the frozen research and scout snapshots, our own team sets and, for search, the usage table and random seed. Suppose the request and protocol prefix supplied at decision t contain only information available to our player by t . Then the neural features and the search agent’s sampled worlds are measurable functions of h_t and those inputs. Two calls with the same request, prefix and inputs produce the same features, and the same distribution over sampled worlds, whatever happens later and whatever remains unrevealed.*

Proof. `observe` and `Tracker.feed` scan only the supplied prefix and merge only the supplied own request. Neither queries the opponent’s private stream or any later log suffix. Feature construction is deterministic in the observation, the candidate string and the fixed inputs. The scout is a fixed function of a filtered public prefix. The search agent samples worlds from a distribution that depends on the tracker state, the usage table and the seed. Composition of measurable maps proves the claim. $\square$

The premise matters. Passing an omniscient log to `observe` would invalidate it. The local simulator bridge therefore observes through the separate `getPlayerStreams().p1/p2` streams; its omniscient stream only drives the simulator [19, 22]. The proposition also does not certify an experiment’s chronology. The checkpoint’s `strategy_knowledge` and the scout are trained on earlier games, and a held-out evaluation must not include games those earlier models were fitted on.

Caveat. Inside each sampled world the search engine is *omniscient* about that world. Both players’ sets are fixed and known to both simulated sides. Non-anticipation holds for the world-sampling step. The per-world matrix game, however, is a perfect-information model of an imperfect-information game. Section 11.7 analyses the bias this introduces.

3.4 How faithful the observation is

The parser covers a selected subset of protocol events. Missing condition data maps to full HP. Under the deployed `strategic-v1` profile, the state encoder still uses the *legacy* HP parser. A coloured public HP string such as 50/100y fails to parse and becomes 1.0. The bug was fixed only for the `tactics-v1`, `rain-v1` and `opening-v1` profiles. In the last 60 ladder games, 16 of 2,509 opposing condition entries carried a colour suffix, affecting 8 of 523 decisions. The strategic prior itself uses the corrected parser, so the defect reaches the network’s learned residual but not its prior. Historical observed opponent moves are stored as `moves_known` and used by the planner; `Features._mon` encodes `moves`. Perspective correctness therefore does not imply a sufficient representation.

Implementation: `observe`, `hp_fraction` and `legacy_hp_fraction` in `battle_state.py`; `Features` in `ml/features.py`; `search/tracker.py`; `simulator.cjs`.

4 Candidate actions and doubles constraints

4.1 Actions are complete player decisions

For an actionable request r , let $\mathcal{A}(r)$ be the list returned by `legal_choices(r)`. In doubles, one candidate is a complete pair $a = (u_1, u_2)$. Each u_i is a move slot with a target, a switch destination or a pass. Examples are

```
move 1 1, move 2 -1
move 1 2 mega, switch 4
```

Positive targets are opposing positions and negative targets are our own. The opponent’s simultaneous choice is *not* part of a candidate. The enumerator takes the Cartesian product of the per-position options. It removes combinations that violate shared constraints, filters disabled or exhausted moves, respects visible trapping, and handles fainted positions. More than two active positions are rejected. Hidden restrictions can still make the server refuse a candidate, so $\mathcal{A}(r)$ is *request-visible admissibility*, not proof of acceptance.

Proposition 4.1 (Constraints enforced by joint enumeration). *Every emitted doubles candidate has distinct switch destinations, at most one Mega-family activation and at most one Terastallization. In a forced-replacement request, the number of switching positions equals $\min(\text{forced positions}, \text{available bench members})$.*

Proof. Each product element is tested in turn. Its switch strings must form a set, so duplicates are rejected. Under forced switching, its switch count must equal the stated minimum. Elements with more than one Mega-family event or more than one Terastallization event are rejected. An emitted element has passed every predicate. $\square$

The proposition covers these predicates only. Champions requests never offer Terastallization, so that branch is dormant. Team validation and other hidden mechanics are outside its scope.

4.2 Team preview and an exact symmetry

With n party members and k to bring, preview candidates are ordered selections, $|\mathcal{A}_{\text{preview}}| = n!/(n-k)!$, which is 360 for six choose four. The first two positions lead; the last two are the bench. Under `opening-v2` and every `strategic-*` profile except `strategic-turn-v1`, the action encoder canonicalizes the bench by sorting its indices before hashing. The representation is therefore exactly invariant to swapping the two bench positions. The actor still scores all 360 candidates, so each plan appears twice.

Proposition 4.2 (Bench symmetry halves the preview space). *Let $\sim$ identify two preview candidates that agree in their ordered leads and in their bench as a set. For six choose four this gives 180 classes of exactly two candidates each. If the features, and hence the logits, are constant on each class, then:*

1. *the class probabilities $\Pi(C) = \sum_{a \in C} \pi(a)$ are a softmax over the 180 class logits, at the same temperature;*
2. *the PPO ratio of any candidate equals the ratio of its class;*
3. *the candidate-level entropy is $H(\pi) = H(\Pi) + \log 2$.*

Proof. There are $6 \cdot 5 = 30$ ordered lead pairs. Each leaves four members, from which $\binom{4}{2} = 6$ bench sets can be formed, so there are 180 classes. Each bench set has exactly two orderings, which accounts for all $180 \cdot 2 = 360$ candidates. Let ℓ_C be the shared logit. Then $\pi(a) = e^{\ell_C/\tau} / \sum_{C'} 2e^{\ell_{C'}/\tau}$ for $a \in C$, so $\Pi(C) = 2\pi(a) = e^{\ell_C/\tau} / \sum_{C'} e^{\ell_{C'}/\tau}$. The ratio $\pi_\theta(a)/\pi_{\text{old}}(a) = \Pi_\theta(C)/\Pi_{\text{old}}(C)$ because the factor 2 cancels. Finally, $H(\pi) = -\sum_C 2\pi_C \log \pi_C = -\sum_C \Pi(C) \log(\Pi(C)/2) = H(\Pi) + \log 2$. $\square$

Interpretation. The duplication is harmless for learning. Ratios are unchanged, and the entropy bonus is shifted by a constant whose gradient is zero. Statements such as “the policy chooses among 360 previews” should still say 180 distinct plans. Lead order is kept on purpose: it fixes which Pokémon occupies the left and right positions.

4.3 Why joint scoring rather than a product policy

Sampling each position independently gives $\pi(u_1, u_2 \mid x) = p(u_1 \mid x)q(u_2 \mid x)$. That family cannot express coordination.

Proposition 4.3 (A coordination a product policy cannot represent). *With two options A, B per position, no unmasked product distribution gives positive mass to both (A, A) and (B, B) and zero mass to both (A, B) and (B, A) .*

Proof. Positive diagonal mass forces $p(A), q(A), p(B), q(B) > 0$, so the off-diagonal products are positive too. Equivalently, every product table has rank one and determinant zero. The required table has determinant $\pi(A, A)\pi(B, B) > 0$. $\square$

The implemented alternative scores each feasible complete candidate directly. Masking a product policy with joint constraints, or factorizing it autoregressively, would also introduce dependence. Feature collisions and finite capacity still limit which joint distributions the network can express.

4.4 Two enumerators

The search engine enumerates candidates separately, inside the simulator, in `sideChoices` of `search/engine.cjs`. Its pruning differs from $\mathcal{A}(r)$:

- Damaging moves aimed at the partner are removed.
- Targets in empty foe slots are removed while the other foe is alive.
- Every move of a Mega-capable Pokémon carries the `mega` flag, so delaying Mega Evolution is never considered.

Our own live choices still start from $\mathcal{A}(r)$ and are pruned in Python. The engine enumeration is used for the opponent and for our forced-replacement look-ahead. Appendix C records a defect in it: when both actives on one side can Mega Evolve, every move-and-move pair is discarded.

Implementation: `legal_choices` in `battle_state.py`; bench canonicalization in `Features.action` (`ml/features.py`); `_prune_ours` in `search/agent.py`; `sideChoices` in `search/engine.cjs`.

5 Features, profiles and the strategic prior

5.1 Fixed-dimensional vectors for variable objects

The state vector has $d_s = 384$ coordinates and each candidate’s action vector has $d_a = 192$. A named feature f with value v_f is added by signed hashing:

$$\phi_j += \sigma(f) v_f, \quad j = H(f) \bmod (d - 16), \quad \sigma(f) \in \{-1, +1\}. \quad (3)$$

Both H and σ come from an eight-byte BLAKE2b digest: the first four bytes, little-endian, give H , and the parity of the fifth byte gives the sign. The last sixteen coordinates are reserved for dense numbers. That leaves 368 hashed state buckets and 176 hashed action buckets. Feature names are namespaced by role and position: species, item, ability, HP, status, boosts, stats, types, moves with their PP ratio, volatiles, field and side conditions. The last sixteen public move events are added with weight $1/\sqrt{i+1}$, counting back from the newest. A research prior adds `meta/<species>` frequencies. It is built from the top 32 team-sheet frequencies, overlaid by the top 32 usage entries. Dense state slots hold turn progress $\min(\text{turn}/50, 2)$, total own HP divided by six, and $|\mathcal{A}|/1000$.

The whole vector is clipped to $[-5, 5]$. After clipping, the decision code adds the scout’s top-three move probabilities (Section 13). The *stored* state therefore can leave the clipped interval.

Proposition 5.1 (Hashing loses distinctions). *A signed hash map from m categorical indicator coordinates into $d' < m$ buckets ($d' = d - 16$ here) is not injective on $\mathbb{R}^m$. Any later function of the hashed vector, including clipping, cannot restore the lost distinctions.*

Proof. The map is linear with rank at most $d' < m$, so its null space is nontrivial. Two inputs that differ by a null-space vector hash to the same output, and any later function maps equal inputs to equal outputs. $\square$

This is a capacity statement, not a claim that two particular battle states collide. Expected-collision results for feature hashing assume random hash functions [33]; a fixed digest over a realized vocabulary does not inherit them automatically.

5.2 Feature profiles

A checkpoint stores a `feature_profile`. Training excludes episodes recorded under any other profile, so a profile is a versioned input semantics. The schema number alone is not enough to identify the inputs. Nineteen profiles exist; Table 3 lists the families.

5.3 The deployed strategic prior

The final action coordinate holds a scalar prior $q_0(x, a)$. Under `strategic-v1` it comes from a one-turn *scenario evaluation* in `ml/strategy.py`, which has four steps.

Opponent hypotheses. For each live opposing active Pokémon, candidate moves are drawn from the disclosed sheet if there is one. Otherwise they are the observed moves plus the five most frequent executed moves for the species in the checkpoint’s `strategy_knowledge`. That table holds counts from 766 earlier games. Each move gets the weight

$$w = 2 + \sqrt{\text{count}} + \min(3, \text{recent}), \quad (4)$$

with Protect multiplied by 1.5 below 40% HP, by 0.65 otherwise, and by 0.3 if it was used on the previous turn. The two strongest attacks and one support move are kept. A single-target attack

Profile family	What it changes
<code>legacy, weather-v1</code>	Original features and the power/type prior of Section 5.4; weather-aware move data.
<code>tactics-v1, rain-v1, mechanics-v1</code>	Damage-based turn priors and rain-team mechanics. <code>tactics-v1</code> and <code>rain-v1</code> , with <code>opening-v1</code> , are the only profiles that use the corrected coloured-HP parser.
<code>preview-*, lineup-v1, opening-*, mega-v1</code>	Preview priors (coverage, lead pressure, exposure) and exact prospective Mega stats under Champions rules.
<code>strategic-v1 (deployed)</code>	Scenario-evaluation prior for turns and previews (Section 5.3); bench canonicalization.
<code>strategic-v2..v5</code>	More mechanics, preview plan summaries, Mega uncertainty. <code>v2-v4</code> were evaluated and rejected; the <code>v5</code> experiment was paused before a completed gate (Section 14).

Table 3: Feature-profile families. Only `strategic-v1` is deployed.

is split over targets: weight 1 goes to the target with the largest damage relative to its remaining HP ($\text{damage}/\max(0.2, h)$), and 0.45 to the others. Two further hypotheses are added when they apply:

- a switch-in from the revealed bench, with weight $0.12 \sum w$;
- a 50/50 Mega-forme split, when the stone is disclosed and the side’s Mega is unspent.

Joint scenarios. Per-position hypotheses are multiplied across the two opposing positions. Impossible combinations, such as two Megas or two switches into one reserve, are removed. The six most probable scenarios are kept and renormalized to probabilities p_k .

Deterministic resolution. For our candidate a and scenario k , `simulate` resolves one turn in priority and speed order, including Trick Room. It accounts for Protect, Wide Guard, redirection, Helping Hand, Fake Out, weather, terrain, Intimidate on entry, drain, recoil and Focus Sash. Damage uses the standard formula

$$\text{dmg} = \left(\frac{22 \text{BP } A/D}{50} + 2 \right) \cdot \text{STAB} \cdot E \cdot \text{burn} \cdot \text{weather} \cdot 0.75^{[\text{spread}]} \cdot 0.925 \cdot \text{acc} / \text{maxHP}, \quad (5)$$

where unknown stats are approximated as base + 35 with stage multipliers, and unknown maximum HP as base HP + 90. The constant 0.925 is the mean damage roll. The outcome value is

$$v_k(a) = \sum_{\text{foes}} [\Delta h + 0.8 \mathbf{1}_{\text{KO}}] - \sum_{\text{own}} [\Delta h(1 + 0.2\rho) + (0.9 + \rho) \mathbf{1}_{\text{KO}}] - 0.08 \# \text{Protect} - 0.10 \# \text{switch} + \dots, \quad (6)$$

where $\rho \in [0, 0.5]$ is a scarcity bonus for a Pokémon that is the only good answer to some opponent. Smaller terms reward a switch-in’s next-turn threat and penalize spending the Mega.

Risk-sensitive aggregation and squashing. If speeds tie, each v_k averages both orders. The turn score is

$$s(a) = 3 \left[(1 - \kappa) \sum_k p_k v_k(a) + \kappa \min_k v_k(a) \right], \quad \kappa = \text{clip}(0.15 + 0.05(R_{\text{own}} - R_{\text{foe}}), 0.05, 0.25), \quad (7)$$

where R_{own} sums our HP fractions. R_{foe} sums the revealed foes’ HP fractions and counts each unseen foe as one. Preview candidates use `opening_score` instead. It takes the better of no-Mega

and one-Mega modes over coverage, lead pressure and exposure, adds terms for rain dependency, Intimidate, Fake Out and Trick Room, and doubles the total. Finally,

$$q_0(a) = g(s(a)), \quad g(s) = \frac{4s}{4 + |s|}. \quad (8)$$

Proposition 5.2 (Properties of the risk aggregate and the squash).

1. For any scenario values, $\min_k v_k \leq (1 - \kappa) \sum_k p_k v_k + \kappa \min_k v_k \leq \sum_k p_k v_k$. The aggregate is monotone in each v_k , and adding a constant c to every v_k adds c to it.
2. The function g is odd, strictly increasing and a bijection from $\mathbb{R}$ onto $(-4, 4)$. It satisfies $0 < g'(s) = 16/(4 + |s|)^2 \leq 1$, so it is 1-Lipschitz. Its ranking of candidates equals the ranking by s .

Proof. (1) Since $\min_k v_k \leq \sum_k p_k v_k$, a convex combination of the two lies between them. Each part is monotone and translation-equivariant, and so is their combination. (2) For $s \geq 0$, $g(s) = 4s/(4+s)$ has derivative $16/(4+s)^2 > 0$; oddness extends this to $s < 0$. The function is continuous at 0, and $g(s) \rightarrow \pm 4$ as $s \rightarrow \pm\infty$. The derivative is largest at $s = 0$, where it equals 1, which gives the Lipschitz bound. A strictly increasing map preserves order. $\square$

Interpretation. The squash replaces a hard clip, which had flattened 288 of the 360 preview candidates into one tied value (`docs/strategic-validation.md`). The pessimism weight κ *increases* when we are ahead on resources: the prior becomes more risk-averse when there is more to lose. The final `clip` to $[-4, 4]$ in the source is redundant after g .

Example 5.3 (A strategic prior, end to end). *Three scenarios have values (0.9, 0.4, -0.6) and probabilities (0.5, 0.3, 0.2), with equal resources, so $\kappa = 0.15$. The expectation is 0.45, the minimum is -0.6, and $s = 3[0.85(0.45) + 0.15(-0.6)] = 0.8775$. The squash gives $q_0 = 4(0.8775)/4.8775 \approx 0.720$. At the deployed temperature $\tau = 0.25$ (Section 6), this prior adds $0.720/0.25 \approx 2.88$ to the effective logit before any learned residual.*

5.4 The legacy power prior

Earlier profiles used the rough pressure term

$$p(m, j) = \frac{\text{BP}(m)}{100} \text{acc}(m) \text{STAB}(m) E(m, j), \quad (9)$$

summed over live targets, negated against allies, minus 0.4 per switch, plus 0.15 for Protect, Detect or Wide Guard, and clipped to $[-4, 4]$.

Example 5.4 (A legacy prior). *Base power 80, accuracy 1, STAB 1.5 and a super-effective multiplier of 2 give $0.8 \cdot 1.5 \cdot 2 = 2.4$. Paired with Protect the candidate scores 2.55; paired with a switch it scores 2.0. These numbers are preferences, not HP damage or win probabilities. At preview this legacy prior is zero; the strategic prior is not.*

The legacy term still feeds a dense action slot, but it is not the deployed prior. Neither prior models full speed control, stat spreads, most items, or the opponent's best response. Search (Section 11) addresses exactly that last omission.

Implementation: `Features.add/state/action` in `ml/features.py`; `hypotheses`, `prepare`, `simulate`, `turn_score` and `opening_score` in `ml/strategy.py`; `damage` in `ml/tactics.py`; `preview_score` in `ml/opening.py`; `species_prior` in `ml/research.py`.

6 The action-conditioned actor-critic

6.1 Exact architecture

Let $x \in \mathbb{R}^{384}$ be the state vector and $z_a \in \mathbb{R}^{192}$ the vector for candidate a . The deployed `concat-v1` network computes

$$e_s = \tanh(W_s x + b_s) \in \mathbb{R}^{128}, \quad e_a = \tanh(W_a z_a + b_a) \in \mathbb{R}^{96}, \quad (10)$$

$$\ell_\theta(x, a) = w_\pi^\top \tanh(W_\pi [e_s; e_a] + b_\pi) + c_\pi + q_0(x, a), \quad (11)$$

$$V_\theta(x) = w_V^\top \tanh(W_V e_s + b_V) + c_V, \quad (12)$$

with an actor hidden width of 96 and a critic hidden width of 64. The prior $q_0(x, a) = z_a[191]$ is the last action coordinate (Section 5). It enters as a fixed residual and has no learned weight. The critic sees only the state. Figure 3 shows the data flow.

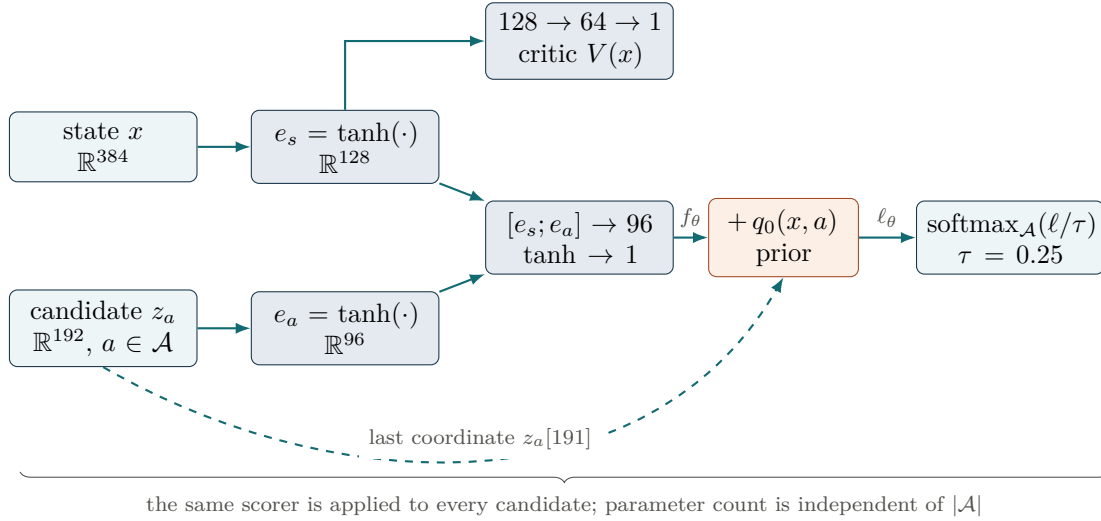


Figure 3: The deployed actor-critic. The final actor layer is initialized to zero, so a fresh actor equals $\text{softmax}(q_0/\tau)$. The optional `matchup-v1` variant adds a bilinear term $u^\top(e_a \odot M e_s)$ to the logit.

Module	Shape	Parameters
State encoder	384 → 128	49,280
Action encoder	192 → 96	18,528
Actor head	224 → 96 → 1	21,697
Critic head	128 → 64 → 1	8,321
concat-v1 total (deployed)		97,826
matchup-v1 extra: M (128 → 96), u (96 → 1)	no bias	12,288 + 96
matchup-v1 total (rejected)		110,210
Scout (Section 13)	384 → 96 → 938	127,946

Table 4: Parameter counts, checked by instantiating the classes. The scout’s output width equals the move vocabulary of the live format.

The `matchup-v1` migration loads every old tensor and zero-initializes u , so migrated logits are exactly unchanged. That variant passed a development panel (70 vs. 55 of 96) but tied its final

panel 126–126 of 200, and was rejected. The deployed checkpoint (revision 893e35e3, 113 updates) is `concat-v1` with profile `strategic-v1`. It uses a feedforward summary of history: no recurrence, no attention, and no search inside the network.

6.2 The tempered masked policy

A checkpoint carries two temperatures in $[0.25, 2]$: one for turns and one for previews. The deployed value is $\tau = 0.25$ for both. For a nonempty candidate set the policy is

$$\pi_\theta(a \mid x, \mathcal{A}) = \begin{cases} \frac{\exp(\ell_\theta(x, a)/\tau)}{\sum_{u \in \mathcal{A}} \exp(\ell_\theta(x, u)/\tau)}, & a \in \mathcal{A}, \\ 0, & a \notin \mathcal{A}. \end{cases} \quad (13)$$

The same τ is used for sampling, for stored log-probabilities, and for the training loss. Inference scores only real candidates. Training pads variable candidate sets and fills padded logits with the most negative finite float. After division by $\tau < 1$ that sentinel overflows to $-\infty$ in float32. The categorical distribution handles this exactly, so padded probabilities are exactly zero.

Proposition 6.1 (Normalization, support and score function). *For finite logits, $\tau > 0$ and a nonempty, parameter-independent set $\mathcal{A}$, the policy (13) sums to one, vanishes outside $\mathcal{A}$, and for $a \in \mathcal{A}$ satisfies*

$$\nabla_\theta \log \pi_\theta(a \mid x, \mathcal{A}) = \frac{1}{\tau} \left(\nabla_\theta \ell_\theta(x, a) - \sum_{u \in \mathcal{A}} \pi_\theta(u \mid x, \mathcal{A}) \nabla_\theta \ell_\theta(x, u) \right). \quad (14)$$

Proof. The denominator is a finite sum of positive terms, so the admissible probabilities are positive and sum to one; the others are zero by definition. The logarithm is $\ell_\theta(x, a)/\tau - \log \sum_u \exp(\ell_\theta(x, u)/\tau)$. Differentiating the log-sum-exp gives the π -weighted average of $\nabla \ell_\theta/\tau$. $\square$

Interpretation. A low temperature multiplies every logit gradient by $1/\tau = 4$ and sharpens the policy. Turn-temperature promotion was itself the project’s largest single measured improvement: the same weights at $\tau = 0.25$ won 129 of 200 paired games against 95 at the previous setting.

Example 6.2 (Temperature changes the same preferences). *For logits (2.55, 2.00, 0.30), $\tau = 1$ gives probabilities (0.594, 0.343, 0.063). The deployed $\tau = 0.25$ gives (0.900, 0.100, 0.0001). Greedy play picks the first candidate in both cases. These numbers are selection probabilities, not win probabilities.*

6.3 The learned residual as a tilt of the prior

Fix an input and candidate set. Split the logit into the prior q_0 and the learned residual $f_\theta = \ell_\theta - q_0$, and let $q_\tau = \text{softmax}(q_0/\tau)$. Then

$$\pi_\theta(a) = \frac{q_\tau(a) e^{f_\theta(a)/\tau}}{Z}, \quad Z = \sum_u q_\tau(u) e^{f_\theta(u)/\tau}. \quad (15)$$

Proposition 6.3 (Variational form with temperature). *For a strictly positive q_τ and finite f on a finite set, the distribution $p^* \propto q_\tau e^{f/\tau}$ is the unique maximizer of $\sum_a p(a) f(a) - \tau D_{\text{KL}}(p \parallel q_\tau)$ over probability vectors p .*

Proof. From $\log p^* = \log q_\tau + f/\tau - \log Z$,

$$\sum_a p(a) f(a) - \tau D_{\text{KL}}(p \| q_\tau) = \tau \log Z - \tau D_{\text{KL}}(p \| p^*).$$

The right side is maximized, uniquely, where the KL divergence vanishes, that is at $p = p^*$. $\square$

Interpretation. The policy behaves as if it maximized the learned residual against a KL penalty toward the tempered prior, with penalty strength τ . At $\tau = 0.25$ the prior is sharpened four-fold and the residual is also amplified four-fold. The trainer never optimizes this objective explicitly, and a large enough residual can overturn the prior completely.

Permutation equivariance. The scorer is applied independently to each candidate. Permuting the candidate rows therefore permutes the logits and probabilities in the same way. With a unique maximum, the greedy choice is independent of list order. Exact ties fall back to the first index.

Implementation: `ActorCritic`, `Model.temperature`, `Model.predict` and `enable_matchups` in `ml/model.py`; `Brain.decide` in `ml/brain.py`.

7 The data recorded for every match

A learning system is only as trustworthy as its records. The recorder answers four questions for every decision:

- What did the player know?
- Which candidates were legal?
- With which probabilities, under which exact checkpoint, was one sampled?
- What happened afterwards?

The current format is *recorder 3*.

7.1 Anatomy of an episode

Figure 4 shows one episode as stored in the `episodes` table. An episode is a JSON document of four parts: a header, the conditions of collection, the decision steps, and a terminal section.

7.2 Tables

The store is one SQLite database in write-ahead-log mode. Its tables are listed in Table 5. Jobs carry a persistent identity, status, attempt count and a lease, which is how the three lanes of Section 9 share work safely.

7.3 What makes a record usable for learning

Before any gradient step, each candidate episode passes `trajectory_issues`. An episode is excluded, not repaired, if any check fails:

1. It is not complete, has another format, revision, schema or feature profile, or its outcome is outside $\{-1, 0, 1\}$.
2. Any ladder step was proposed but never submitted.

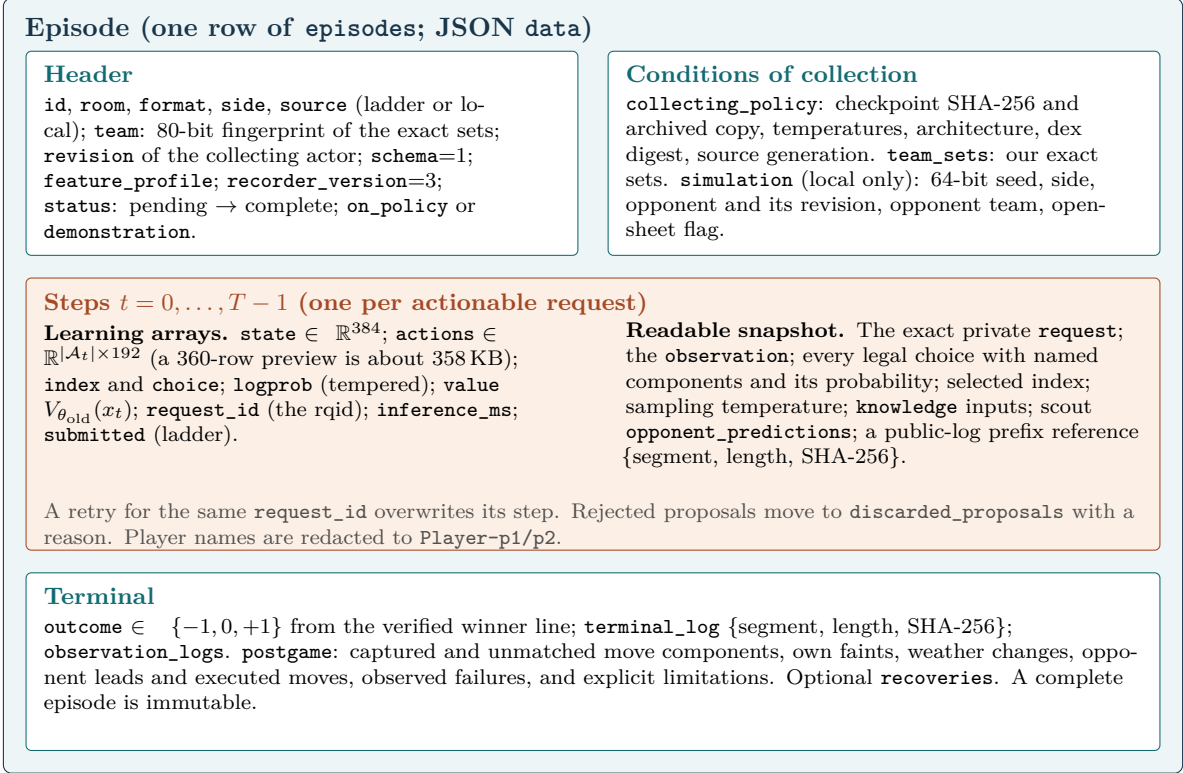


Figure 4: The recorder-3 episode. Learning uses the orange arrays; auditing and replay use the snapshot. For the last 20 ladder games the median document was 1.30 MB (range 0.73–2.06 MB). In a 7-step game, feature arrays took 750 KB and snapshots 440 KB of 1.20 MB.

Table	Contents	Rows (M-C)
episodes	Header columns plus the JSON document above	14,137 complete
jobs	Collector, learner, feed and evaluator jobs with leases	—
public_battles	Sanitized public logs, unique by digest	12,573
scout_samples	Pre-turn public vectors with executed-move labels	192,835
research_teams, documents	Attributed tournament sets and prose	—
team_metadata, feed_state	Team versions; daily feed bookkeeping	—

Table 5: SQLite schema. Counts are a read-only snapshot taken on 9 October 2026 at about 20:15 UTC, while collection was still running (database 18.4 GB). Of the complete M-C episodes, 854 are recorded ladder games (353–501) and 13,283 are local self-play games (9,010–4,273); there are no ties. The recorded ladder count also includes browser games played in model mode after the 847-game campaign stopped.

3. Any array has the wrong shape or a non-finite value, or any stored log-probability exceeds 10^{-6} .
4. The snapshot’s probabilities do not sum to one, or $p[\text{index}] \neq e^{\log\text{prob}}$ (relative tolerance 10^{-5}).
5. The snapshot’s choices differ from `legal_choices` of its own stored request.
6. A log-prefix digest fails, a step contains future or terminal lines, a request is credited twice, or the terminal log lacks a win or tie line.

A second check recomputes every stored log-probability under the archived collecting checkpoint (Section 8.5). Excluded episodes are never marked consumed, so a persistent defect stays visible in later batch reports rather than disappearing silently.

7.4 Games played by search

A browser game decided by search writes no `episodes` row. Its actions were not sampled from π_θ , so storing them as PPO samples would give false importance ratios (Section 8.4). The runner logs each decision, with its request, choice and search statistics, to a private per-run ledger. At the end of the game, the sanitized public log enters `public_battles` as an `own-live-game` and yields scout labels.

7.5 Storage lifecycle and budget

Data has a bounded lifetime, shown in Figure 5. Each candidate freezes the inference inputs it was evaluated with: research teams, documents, species and move support, and scout weights. These are stored as a content-addressed, read-only snapshot. Completed snapshots are gzip-archived every 900 s, with byte-count and SHA-256 verification, so they stay reproducible. A storage budget limits optional work.

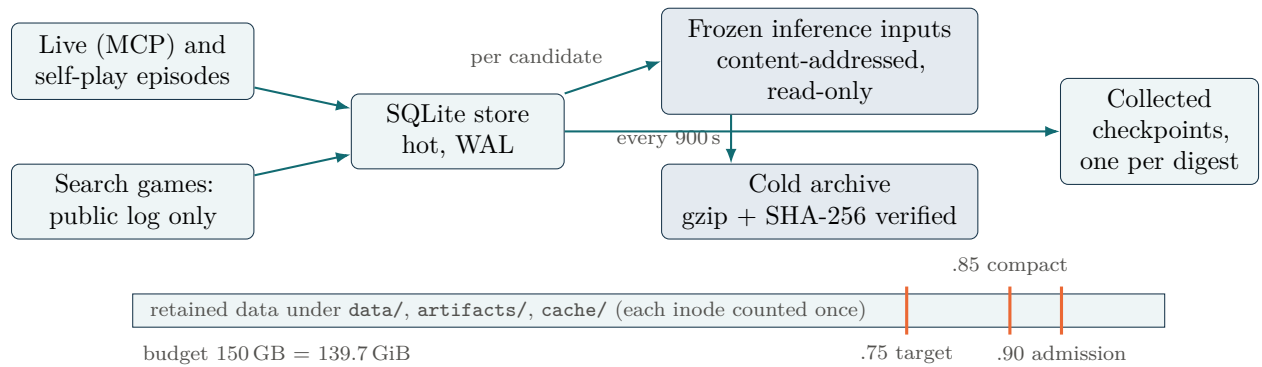


Figure 5: Data lifecycle and the storage budget. Above 90% of the budget, collection, learning and evaluation defer. Above 85%, finished candidates’ input snapshots are compacted down to 75%. Collecting checkpoints are kept once per digest, so every recorded likelihood can be recomputed.

Implementation: `ml/storage.py` (schema, fingerprint, collected checkpoints); `ml/brain.py` and `ml/recording.py` (steps and snapshots); `ml/postgame.py`; `ml/data_quality.py`; `ml/input_snapshot.py`; `ml/data_budget.py`; `docs/cold-input-archiving.md`; `docs/model-storage.md`.

8 Learning from outcomes: policy gradients, GAE and PPO

8.1 Reward and objective

For a completed episode with T recorded decisions, let $Y \in \{-1, 0, 1\}$ be loss, tie or win from our side. The reward is terminal only: $r_t = 0$ for $t < T - 1$ and $r_{T-1} = Y$. With $\gamma = 0.99$,

$$G_t = \sum_{j=t}^{T-1} \gamma^{j-t} r_j = \gamma^{T-1-t} Y. \quad (16)$$

With $\gamma = 1$, $\mathbb{E}[Y] = \mathbb{P}(\text{win}) - \mathbb{P}(\text{loss}) = 2\mathbb{P}(\text{win}) + \mathbb{P}(\text{tie}) - 1$. When ties are absent, as in every recorded M-C game, maximizing $\mathbb{E}[Y]$ is the same as maximizing win probability. With $\gamma < 1$, the same outcome is worth less after a longer game, so training mildly prefers shorter wins and longer losses. The promotion gate counts raw wins. The two criteria are related but not identical.

For a fixed policy, the history-conditioned value, action value and advantage are $V^\pi(h_t) = \mathbb{E}_\pi[G_t \mid h_t]$, $Q^\pi(h_t, a) = \mathbb{E}_\pi[G_t \mid h_t, a_t = a]$ and $A^\pi = Q^\pi - V^\pi$. The critic approximates V^π through compressed features. Its output is unbounded and is not a calibrated win probability.

8.2 The policy-gradient identity on histories

Theorem 8.1 (Finite-horizon likelihood-ratio gradient). *Assume a finite trajectory space and a finite horizon T , padding terminated trajectories with zero-reward absorbing steps. Assume bounded rewards, a differentiable policy with parameter-independent support on each history, and environment and opponent kernels that do not depend on θ . Let $J(\theta) = \mathbb{E}_\theta[\sum_{t<T} \gamma^t r_t]$. Then*

$$\nabla_\theta J(\theta) = \mathbb{E}_\theta \left[\sum_{t=0}^{T-1} \gamma^t G_t \nabla_\theta \log \pi_\theta(a_t \mid h_t) \right], \quad (17)$$

and G_t may be replaced by $G_t - B(h_t)$ for any action-independent baseline B without changing the expectation.

Proof. A trajectory’s probability factors into initial, transition and policy terms. Only the policy terms depend on θ , so $\nabla \log p_\theta(\tau) = \sum_t \nabla \log \pi_\theta(a_t \mid h_t)$. Differentiating the finite expectation gives $\mathbb{E}[R(\tau) \nabla \log p_\theta(\tau)]$. A reward earned before a_t is determined by h_t , and $\mathbb{E}[\nabla \log \pi_\theta(a_t \mid h_t) \mid h_t] = \sum_a \nabla \pi_\theta(a \mid h_t) = \nabla 1 = 0$. Those cross terms therefore vanish, which leaves $\sum_{j \geq t} \gamma^j r_j = \gamma^t G_t$. The same zero-mean identity removes the baseline. $\square$

Interpretation. An action gains probability when the outcome that follows it is better than expected for that situation. The theorem needs no Markov assumption on the hashed features, because it is stated on histories. It does need the recorded action to have actually been sampled from π_θ . This is why search decisions are never PPO data.

Caveat. The implemented surrogate averages decision samples equally and omits the γ^t factor. It is therefore a standard heuristic related to Equation (17), not an unbiased estimator of ∇J [26, 27, 30]. Opponent kernels are not fixed in self-play either: the `self` opponent is a frozen snapshot of the incumbent, which is fixed within a batch but changes after each promotion.

8.3 Generalized advantage estimation

Using the collecting critic's stored values V_t , and $V_T = 0$,

$$\delta_t = r_t + \gamma V_{t+1} - V_t, \quad \hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}, \quad \hat{A}_T = 0, \quad \lambda = 0.95, \quad (18)$$

$$\hat{R}_t = \hat{A}_t + V_t. \quad (19)$$

Figure 6 shows how the recursion runs backward through one episode.

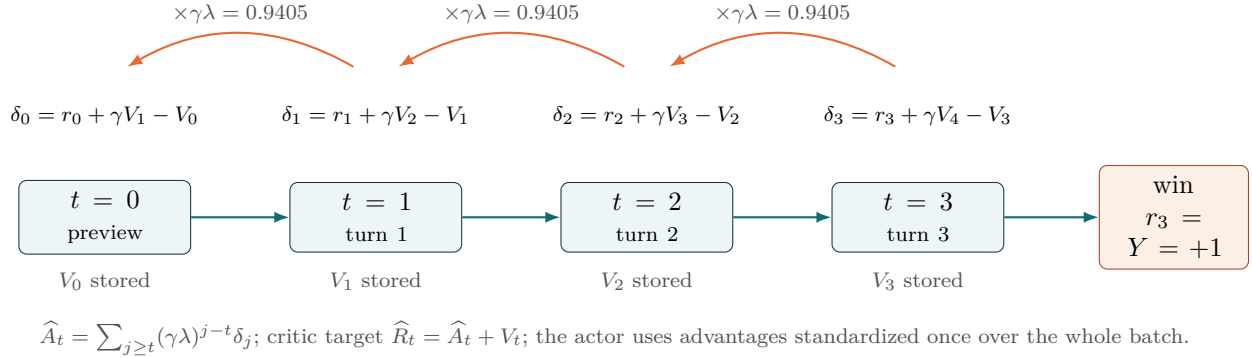


Figure 6: GAE on a four-decision episode with a terminal win. Only the last residual contains the reward. Each earlier advantage adds its own residual to the next advantage, discounted by $\gamma \lambda$.

Proposition 8.2 (GAE expansion and the Monte Carlo endpoint). *The recurrence (19) gives $\hat{A}_t = \sum_{j=t}^{T-1} (\gamma \lambda)^{j-t} \delta_j$. If $\lambda = 1$ and $V_T = 0$, then $\hat{A}_t = G_t - V_t$.*

Proof. Unroll the recurrence backward from $\hat{A}_T = 0$, or argue by induction from $t = T - 1$. With $\lambda = 1$, the $+\gamma V_{j+1}$ in each residual cancels the $-V_{j+1}$ in the next one after discounting. What remains is the discounted rewards, $-V_t$, and $\gamma^{T-t} V_T = 0$. $\square$

Example 8.3 (Two decisions ending in a win). *With $(V_0, V_1) = (0.20, 0.30)$ and $(r_0, r_1) = (0, 1)$: $\delta_1 = 0.70$, $\delta_0 = 0.99(0.30) - 0.20 = 0.097$, $\hat{A}_1 = 0.70$, and $\hat{A}_0 = 0.097 + 0.9405(0.70) = 0.75535$. The critic targets are $(0.95535, 1.00)$; the Monte Carlo target for $t = 0$ would be 0.99 .*

8.4 Stored probabilities and importance ratios

For a step collected by revision θ_{old} , PPO uses

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | x_t, \mathcal{A}_t)}{\pi_{\theta_{\text{old}}}(a_t | x_t, \mathcal{A}_t)} = \exp(\log \pi_\theta(a_t | x_t, \mathcal{A}_t) - \text{storedLogProb}_t), \quad (20)$$

with both policies tempered as in Equation (13).

Proposition 8.4 (Conditional importance identity). *At a fixed stored input and candidate set, if the old policy p is positive wherever the new policy q is, then $\mathbb{E}_{a \sim p}[(q(a)/p(a))f(a)] = \mathbb{E}_{a \sim q}[f(a)]$ for every real f .*

Proof. Write the left side as $\sum_a p(a) q(a) f(a) / p(a)$ and cancel $p(a)$ on its support. The support assumption means no term with positive q -mass is lost. $\square$

This corrects the action choice at fixed inputs only, not the whole trajectory distribution. It is wrong when the action was really chosen by greedy play, a human, search, or an unknown policy. Rejections add a further subtlety. The stored likelihood is the probability of the *proposal*. If hidden restrictions reject some proposals and only accepted ones survive, the executed action follows the proposal policy conditioned on acceptance. The recorder therefore keeps rejected proposals separately, and accepts a step only after its submission is confirmed.

8.5 Collecting-likelihood validation

Before training, the learner reloads the parent (collecting) weights and recomputes every stored step’s tempered log-probability. Any episode with a step where the recomputed and stored values differ by more than $\eta = 10^{-4}$ is excluded.

Proposition 8.5 (What the likelihood check guarantees). *Suppose every retained step satisfies $|\log \pi_{\theta_{\text{old}}}(a_t) - \text{storedLogProb}_t| \leq \eta$, with the left side recomputed from the stored inputs. Then for every θ the ratio used in training equals the exact ratio times e^{ϵ_t} with $|\epsilon_t| \leq \eta$. For $\eta = 10^{-4}$, the relative ratio error is at most $e^\eta - 1 \approx 1.00005 \times 10^{-4}$.*

Proof. Write $\epsilon_t = \log \pi_{\theta_{\text{old}}}(a_t) - \text{storedLogProb}_t$. Then $\exp(\log \pi_\theta - \text{storedLogProb}) = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})e^{\epsilon_t}$, and $|\epsilon_t| \leq \eta$ by assumption. $\square$

Interpretation. The check certifies more than arithmetic. A trajectory whose features, candidate list, checkpoint or temperature changed after recording would fail it. The six most recent learner jobs had a maximum discrepancy of 1.9×10^{-6} . The check cannot certify that the recorded inputs were themselves a faithful observation; Section 3 covers that separately.

8.6 The optimized PPO loss

Advantages are standardized once over the whole batch, before any epoch: $\tilde{A}_t = (\hat{A}_t - \bar{A}) / (s_A + 10^{-8})$ when the batch has more than one sample and $s_A > 10^{-6}$. Here s_A is the population standard deviation. Each minibatch $\mathcal{B}$ of size B also carries phase weights $\omega_t = w_t / \bar{w}_{\mathcal{B}}$, where w_t is the preview training weight ($1 \leq w \leq 16$) for preview steps and 1 otherwise. The minimized loss is

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{t \in \mathcal{B}} \omega_t \left[-\min(\rho_t \tilde{A}_t, \text{clip}(\rho_t, 0.8, 1.2) \tilde{A}_t) + 0.5(V_\theta(x_t) - \hat{R}_t)^2 - c_H H(\pi_\theta(\cdot | x_t, \mathcal{A}_t)) \right]. \quad (21)$$

The entropy is that of the tempered distribution, and the value loss is not clipped. The deployed configuration is: entropy coefficient $c_H = 0.01$; preview weight 1 (so $\omega_t \equiv 1$); Adam with learning rate $3 \cdot 10^{-4}$; minibatches of 32 drawn by a seeded permutation; four epochs; global gradient-norm clip 0.5; target KL 0.03. A non-finite loss aborts before any checkpoint is written. Each training call produces a new revision UUID and increments the update counter once.

Proposition 8.6 (What clipping bounds). *For a fixed advantage A and positive ratio ρ ,*

$$\min(\rho A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon)A) = \begin{cases} A \min(\rho, 1 + \epsilon), & A \geq 0, \\ A \max(\rho, 1 - \epsilon), & A < 0, \end{cases}$$

and this is never larger than the unclipped term ρA .

Proof. For $A \geq 0$ multiplication by A preserves order, so the minimum picks the smaller of ρ and $\text{clip}(\rho)$. That is $\min(\rho, 1 + \epsilon)$, because clipping from below never lowers ρ . For $A < 0$ the order reverses. The minimum picks the larger of the two, which is $\max(\rho, 1 - \epsilon)$. Taking a minimum with ρA cannot exceed ρA . $\square$

Example 8.7 (A clipped update). *If $\pi_{\text{old}}(a) = 0.20$, $\pi_{\theta}(a) = 0.30$ and $A = 0.70$, then $\rho = 1.5$. The unclipped term is 1.05; the clipped term is $1.2(0.70) = 0.84$. With $A = -0.70$ and $\rho = 0.5$ the clipped term is $0.8(-0.70) = -0.56$, not -0.35 .*

The proposition does not keep every ratio within $[0.8, 1.2]$, does not enforce a KL constraint, and does not lower-bound win-rate improvement [25, 27]. The KL stop below is the implementation’s coarse trust region.

8.7 The KL early stop

After each full epoch, the learner evaluates the whole batch at the current parameters and computes

$$\widehat{D_{\text{KL}}} = \frac{1}{N} \sum_{i=1}^N [(\rho_i - 1) - \log \rho_i], \quad (22)$$

where ρ_i is the ratio at the stored action. If $\widehat{D_{\text{KL}}} > 0.03$, the remaining epochs are skipped. The updates from the epoch that crossed the threshold are *kept*.

Proposition 8.8 (The KL estimator). *Fix a state, an old policy p and a new policy q with the same support, and let $\rho(a) = q(a)/p(a)$. Then each term $(\rho - 1) - \log \rho$ is non-negative, and $\mathbb{E}_{a \sim p}[(\rho(a) - 1) - \log \rho(a)] = D_{\text{KL}}(p||q)$. Consequently, if the stored actions were sampled from the old policy, Equation (22) is an unbiased estimate of the average $D_{\text{KL}}(\pi_{\text{old}}||\pi_{\theta})$ over the batch’s stored states.*

Proof. For $u > 0$, $u - 1 - \log u \geq 0$, with equality only at $u = 1$, because $\log u \leq u - 1$. Next, $\mathbb{E}_p[\rho] = \sum_a q(a) = 1$, so $\mathbb{E}_p[\rho - 1] = 0$, and $\mathbb{E}_p[-\log \rho] = \sum_a p(a) \log(p(a)/q(a)) = D_{\text{KL}}(p||q)$. Averaging over states with their stored actions preserves unbiasedness, by linearity. $\square$

Caveat. The estimator is unbiased for $D_{\text{KL}}(\text{old}||\text{new})$ (24). The check runs only between epochs, so the returned candidate can exceed 0.03 by up to one epoch’s movement. Recent production batches stopped at final-epoch values between 0.012 and 0.017, so none triggered the stop. The statistic is still computed on the same samples used for the gradient, so it is a training-set diagnostic rather than a held-out one.

8.8 Demonstrations

For explicitly labelled demonstrations $(x_t, \mathcal{A}_t, a_t^*)$, a separate training call minimizes

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{B} \sum_t \omega_t \log \pi_{\theta}(a_t^* | x_t, \mathcal{A}_t), \quad (23)$$

the weighted conditional log-likelihood. There is no ratio, entropy or value term, and no collecting-likelihood check. The shared state encoder still changes, so the critic moves too. Imitating a weak teacher does not make the learner an expert.

Implementation: `Model.train` (eligibility, GAE, likelihood validation, standardization, loss, KL stop, saving) in `ml/model.py`; `RolloutBatcher` in `ml/batching.py`; `trajectory_issues` in `ml/data_quality.py`; `LearningConfig` in `ml/continuous.py`.

9 The training pipeline on CPU and Apple MPS

9.1 From a game to a deployed checkpoint

Figure 7 traces one unit of experience through the pipeline. Each arrow is a durable hand-off: a SQLite row, a job, a file written by atomic replacement, or a lease. A crash at any point therefore leaves either the old state or the new one, and the next supervisor tick resumes from it.

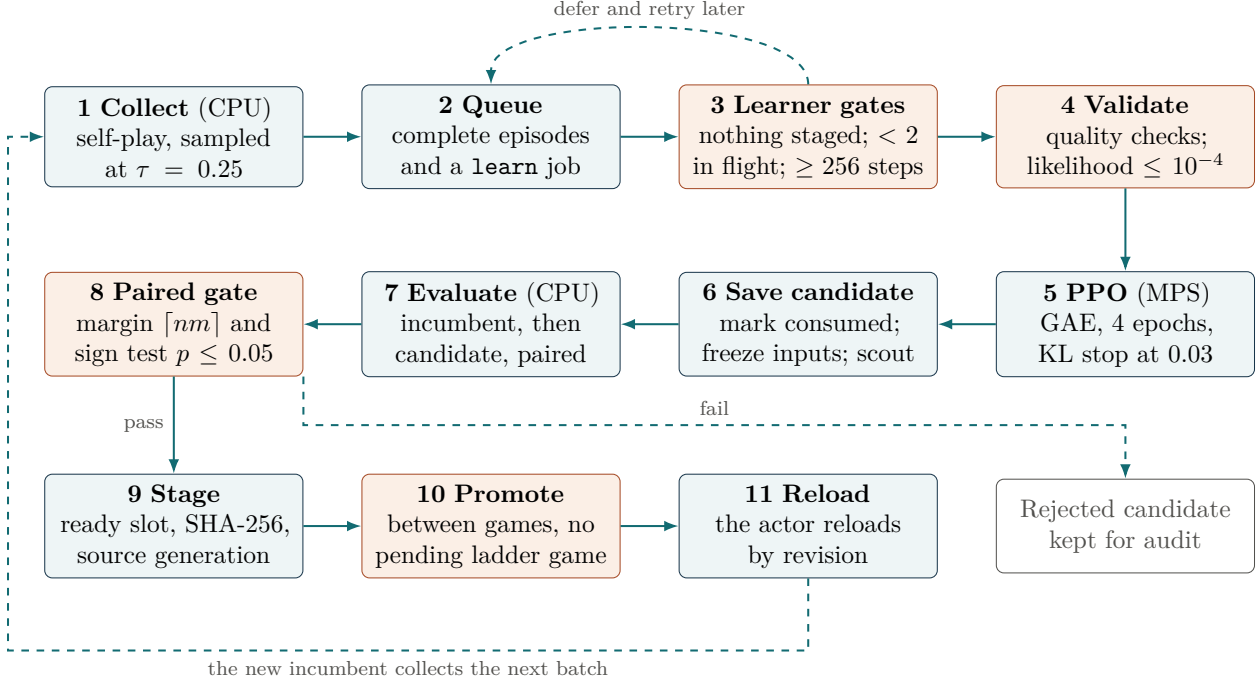


Figure 7: The lifecycle of experience. Orange boxes are gates that can stop the flow. Steps 1 and 7 run on the CPU, step 5 on the MPS GPU. Promotion (step 10) is called by the player between games, never by a worker. Episodes are marked consumed when the candidate is saved, before it is evaluated, so “trained” means *used for a candidate*, not *deployed*.

Collection. The collector snapshots the incumbent and freezes its inference inputs. It then plays `simulation_games` games (default 24) in waves of spawned one-thread processes. Each process drives the official `BattleStream` through `simulator.cjs` and observes only its own player stream. The `ladder-v1` curriculum cycles the opponent through

(self, tactical, heuristic, self, tactical, random, heuristic, tactical),

with index $(\lfloor i/|\text{teams}| \rfloor + \text{seed}) \bmod 8$, so one opponent completes a sweep of the team pool before the next takes over. In each cycle that is 2/8 self-play against the frozen incumbent, 3/8 tactical, 2/8 heuristic and 1/8 random. Open team sheets are drawn with probability 0.08. Sides alternate as $(i + \lfloor i/|\text{teams}| \rfloor) \bmod 2$. Seeds are 64-bit, split into four 16-bit words. An earlier encoding used only 16 bits and was retired.

Learning. The learner applies the gates of step 3, the validation of step 4 and the loss of Section 8. It runs on the device chosen by `backend_for`. In automatic mode MPS is chosen only if its benchmarked time is at most 0.85 of the CPU time. Any device error retrains the same batch from

the incumbent on the CPU. For the strategic profiles the learner then refreshes the candidate’s embedded `strategy_knowledge` with new ladder postgame counts. It also trains the public-move scout on the same lane.

Evaluation. Incumbent and candidate are frozen CPU snapshots. They play n games each (60 in the deployed configuration; default 20), on seeds $s_0 + 100000 + i$ derived from the job identity. Actions are sampled at the checkpoint temperature. Both panels face the same frozen `self` snapshot and the same frozen inputs. Panels resume after interruption. The evaluator abandons a plan if the source generation, parent revision or candidate hash has changed.

9.2 Three lanes on one laptop

Figure 8 shows how the work is shared out in time. A supervisor under launchd `KeepAlive` wakes every 15s. It reloads the autopilot settings, samples resources, enqueues daily feeds and collection, and starts at most one child per lane, running `python -m ml.worker -lane` with the lane name (`collector`, `learner` or `evaluator`). Each child takes its own lock and claims jobs atomically with `BEGIN IMMEDIATE`. A job’s lease is its time budget plus 60s. A job can be claimed while its attempt count is below three, and a deferral gives its attempt back, so deferrals do not exhaust a job. A child exits after its time budget, and the next tick replaces it.

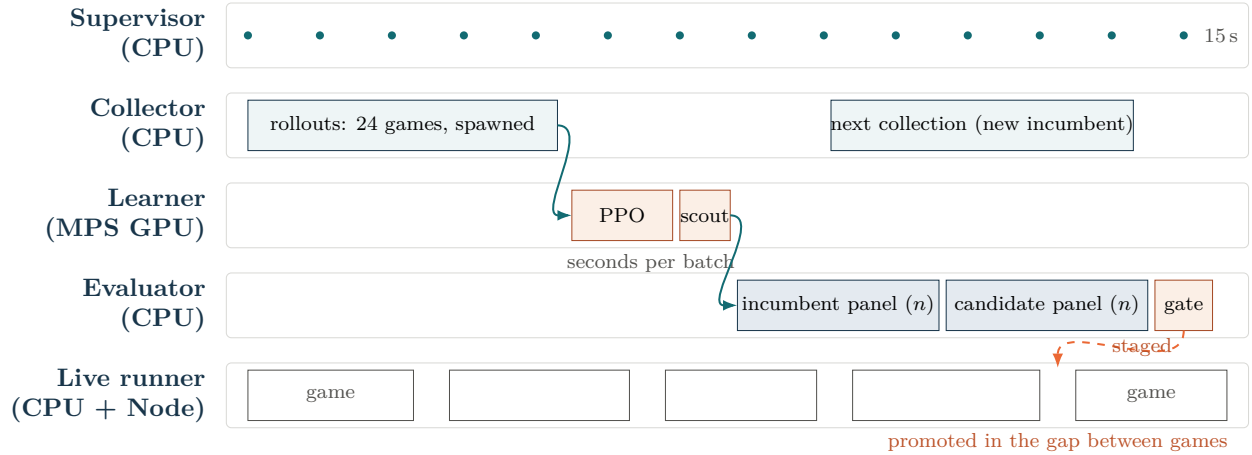


Figure 8: Schematic timeline of the lanes; durations are not to scale. The collector and evaluator share one pool of simulator slots. Within the default budget of four, the evaluator gets half (calibrated locally to three) and the collector the rest. The learner owns the GPU. The live runner is outside the pipeline and only ever consumes a staged candidate at a game boundary.

9.3 Resource admission

Every lane re-samples the host before starting work and at minibatch checkpoints. With C logical cores, a measured CPU load of u percent, available memory F GiB, r resident workers and an estimated m MiB per worker, the allowed simulator worker count is

$$W_{\text{mem}} = \max(0, \lfloor (F - F_{\min})1024/m \rfloor + r), \quad W_{\text{cpu}} = \max(0, \lfloor (u_{\max} - u)C/100 \rfloor), \quad (24)$$

$$W = \mathbf{1}[\text{healthy}] \cdot \max(0, \min[W_{\max}, C - C_{\text{res}}, W_{\text{mem}}, W_{\text{cpu}}]), \quad (25)$$

where *healthy* means $F \geq F_{\min}$, $u < u_{\max}$, swap-out below its limit, no critical memory pressure (macOS `kern.memorystatus_vm_pressure_level` flag value 4), and at least 2 GiB of free disk. “Yellow” pressure (flag value 2) is allowed; only “red” blocks. The code defaults are $C_{\text{res}} = 3$, $F_{\min} = 2$ GiB, $W_{\max} = 4$, $u_{\max} = 75\%$, $m = 400$ MiB and swap-out 32 MiB/s. The deployment uses a modestly higher CPU ceiling and a calibrated swap limit. Optional work also stops above 90% of the storage budget (Figure 5).

Proposition 9.1 (Headroom invariants). *Whenever $W > 0$:*

1. *at most $C - C_{\text{res}}$ simulator workers run, so C_{res} cores stay free of simulator workers;*
2. *if each new worker uses at most m MiB, launching $W - r$ of them leaves at least $F_{\min}$ GiB available;*
3. *if each worker adds at most $100/C$ percentage points of load (one core), the projected load $u + 100W/C$ stays at or below $u_{\max}$.*

Proof. (1) $W \leq C - C_{\text{res}}$. (2) $W - r \leq \lfloor (F - F_{\min})1024/m \rfloor$, so $(W - r)m/1024 \leq F - F_{\min}$. (3) $W \leq \lfloor (u_{\max} - u)C/100 \rfloor$ gives $100W/C \leq u_{\max} - u$. $\square$

Interpretation. The memory term credits resident workers, whose memory is already counted in F . The CPU term does not credit them, even though their load is already in u . It is therefore conservative and can under-admit while workers are running. These are cooperative limits enforced by the application, not OS quotas. Another service can still take the headroom between samples, and the 5-second re-sampling only bounds how long that can go unnoticed.

9.4 The MPS lane

The learner calls `torch.mps.set_per_process_memory_fraction(0.12)`, which caps its Metal allocations at 12% of the device’s recommended working-set size [23]. Its thread count is limited and it runs at `nice 10`. A GPU duty fraction $d \in (0, 1]$ throttles training. After each optimizer step the learner synchronizes with Metal, measures the step’s busy time e_k , and sleeps

$$s_k = e_k \left(\frac{1}{d} - 1 \right). \quad (26)$$

Proposition 9.2 (Duty cycle). *Under this rule each step, and therefore any union of steps, has busy fraction exactly d : $\sum_k e_k / \sum_k (e_k + s_k) = d$.*

Proof. $e_k + s_k = e_k/d$, so $\sum_k (e_k + s_k) = \sum_k e_k/d$. $\square$

The proposition concerns wall-clock time inside the training loop, measured after synchronization. It is not a statement about instantaneous GPU utilization, which the application cannot observe. The default is $d = 0.5$. For these small cached batches, half duty cost several times the full-duty time: 34.99 vs. 7.45 ms per update in Table 6, which is more than the factor of two Proposition 9.2 accounts for. The deployment therefore runs at full duty and relies on the memory cap and the admission guards (`docs/offline-ppo-followup.md`).

9.5 The tensor cache

Rebuilding Python feature lists dominated training time, so batches are converted to tensors once. With N steps and at most C candidates per step, the resident footprint in float32 is $N(384 \cdot 4 + C(192 \cdot 4 + 1) + 8)$ bytes. The extra byte per candidate is its mask entry and the 8 bytes per step hold the index. The batch is kept on the device if this is at most 2^{27} bytes (128 MiB). Otherwise a bounded CPU LRU cache transfers minibatches one at a time.

Example 9.3 (When a batch stays resident). *A batch containing a 360-candidate preview has $C = 360$, so each step costs $1536 + 360 \cdot 769 + 8 = 278,384$ bytes. The residency limit is $\lfloor 2^{27} / 278,384 \rfloor = 482$ steps. Recent production batches of 404 and 415 steps were resident; a 617-step batch used the CPU cache.*

64-decision batch, up to 360 choices	Before cache (ms/update)	After cache (ms/update)
CPU	43.08	14.08
MPS, 50% duty	121.61	34.99
MPS, full duty	44.06	7.45

Table 6: Bounded M5 timing windows from `docs/m5-learning-orchestration-research.md`. They were measured while other workloads ran, so they are diagnostic timings rather than randomized comparisons. A raw kernel benchmark favoured MPS by only 1.057 vs. 1.512 ms; most of the speedup came from caching.

9.6 Safe reloads of source code

The *source generation* is a SHA-256 over the entry points, `simulator.cjs`, the lock file and all `ml/*.py` files. Rollouts refuse to run under a mismatched generation. On a change, the supervisor stops starting children, drains the running ones and exits with code 75 so that `launchd` restarts it under the new code. Evaluation plans and staged candidates record their generation, and promotion refuses a candidate evaluated under different code. A source change therefore invalidates evaluations; it never mixes code versions inside one comparison. On 8–9 October, 25 evaluations were abandoned for this reason and two because the parent had advanced.

Implementation: `ml/pipeline.py` (supervisor, slots), `ml/worker.py` (lanes, collection, learn, evaluate), `ml/resources.py` (ResourcePolicy, BackgroundGuard, backend_for), `ml/batching.py`, `ml/reload.py`, `ml/simulator.py`, `simulator.cjs`, `scripts/learning_service.py`.

10 Promotion: a paired test and what it can certify

10.1 The paired design

For game i the incumbent and the candidate face identical conditions: the same seed, side, opponent, opponent revision, learner team, opponent team, open-sheet flag, seed encoding and effective simulator seed. The gate rejects the run if any of these differ between the two panels. Let $X_i^I, X_i^C \in \{0, 1\}$ be the incumbent’s and the candidate’s wins in pair i . Pairs with $X_i^I = X_i^C$ are *concordant* and carry no information about which policy is better. The informative pairs are the gains ($X^C = 1, X^I = 0$), counted by g , and the losses, counted by ℓ . The discordant total is $d = g + \ell$, and $W_C - W_I = g - \ell$. Figure 9 shows the idea.

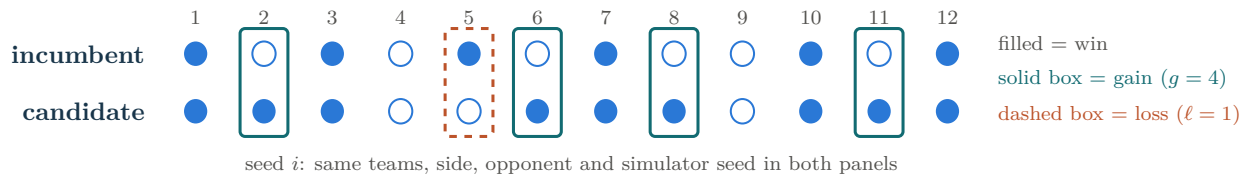


Figure 9: The paired design on twelve illustrative seeds. Only the five discordant seeds bear on the comparison. Here $g = 4$, $\ell = 1$ and $p = \mathbb{P}(\text{Bin}(5, \frac{1}{2}) \geq 4) = 0.1875$, so this run would fail.

10.2 The gate

With n games per policy and margin m , a candidate passes when

$$\text{clean} \wedge (W_C - W_I \geq \lceil nm \rceil) \wedge p \leq 0.05, \quad p = \sum_{k=g}^d \binom{d}{k} 2^{-d} \quad (p = 1 \text{ if } d = 0). \quad (27)$$

Clean means: the run is complete and fully paired; there are no unfinished games and no rejected actions; and every game has a winner or a tie. The defaults are $n = 20$, $m = 0.10$; the deployment uses $n = 60$, $m = 0.05$. The first edition’s gate formula, $\max(1, \lfloor nm + 0.999 \rfloor)$ with no test, is obsolete.

Proposition 10.1 (Validity of the gate’s test). *Suppose the pairs are independent, and under the null hypothesis each discordant pair is equally likely to be a gain or a loss: $\mathbb{P}(X^C = 1, X^I = 0) = \mathbb{P}(X^C = 0, X^I = 1)$ for every pair. Then $\mathbb{P}(p \leq \alpha) \leq \alpha$ for every $\alpha \in [0, 1]$. In particular, the gate passes a candidate that is no better than the incumbent with probability at most 0.05.*

Proof. Condition on which pairs are discordant, and hence on d . Under the null, independence and the equal-probability condition make each discordant pair a gain with probability $\frac{1}{2}$, independently. So $g \mid d \sim \text{Bin}(d, \frac{1}{2})$. The statistic p is the upper-tail probability of this exact distribution at the observed g . By the standard property of discrete tail probabilities, $\mathbb{P}(p \leq \alpha \mid d) \leq \alpha$. Averaging over d preserves the bound. The other gate conditions only remove passes. $\square$

This is the exact conditional form of McNemar’s test [13]. It needs neither a normal approximation nor equal marginal rates across pairs. Its null is “no systematic difference on the evaluated distribution”, not “no difference anywhere”.

Discordant pairs d	Minimum gains g	Minimum net $g - \ell$	p at that g
4 or fewer	—	impossible	≥ 0.0625
5	5	5	0.0313
8	7	6	0.0352
10	9	8	0.0107
15	12	9	0.0176
20	15	10	0.0207
30	20	10	0.0494
40	26	12	0.0403

Table 7: What the sign test demands. With $n = 60$, $m = 0.05$ the margin requires a net of only $\lceil 3 \rceil = 3$, so the test is always the binding condition. The one production pass on 9 October was $38 \rightarrow 46$ wins with $g = 9$, $\ell = 1$, $p = 0.0107$.

Caveat. The source computes $\lceil nm \rceil$ in floating point. For some pairs this exceeds the exact ceiling by one; for example $100 \times 0.07 = 7.000000000000001$ rounds up to 8. The configurations actually used, (20, 0.10), (60, 0.05), (96, 0.10), (192, 0.10) and (200, 0.05), are unaffected. A rational or integer form would remove the edge case.

10.3 Many gates: multiplicity and the winner’s curse

One valid test is not the same as a valid campaign of tests. Between 8 October 18:18 UTC and 9 October 19:57 UTC the pipeline completed 129 paired 60-game gates, and one passed.

Proposition 10.2 (Expected false promotions). *Run K gates, each with type-I error at most α under its null. If every candidate is null, the expected number of passes is at most $K\alpha$, and the probability of at least one pass is at most $\min(1, K\alpha)$. Testing each at α/K (Bonferroni) bounds the family-wise error by α .*

Proof. The expected number of passes is the sum of the individual pass probabilities, each at most α under the null. The union bound gives the second claim and, with α/K , the third. $\square$

With $K = 129$ and $\alpha = 0.05$, up to 6.45 false passes are possible even if no candidate is better, so a single pass is not evidence that PPO is improving the actor. The exact sign test is discrete and conservative, so the true expectation is somewhat lower, but the conclusion stands. A Bonferroni threshold would be $0.05/129 \approx 3.9 \times 10^{-4}$; the observed $p = 0.0107$ does not meet it. The project’s own history shows the *winner’s curse* directly. `matchup-v1` passed development ($p = 0.0068$) and then tied its independent final 126–126. `strategic-v4` reached $p = 0.008$ on a development panel and then went 139–136 on a fresh confirmation ($p = 0.39$). Selection on one panel inflates that panel’s apparent effect. An independent, prospectively specified final panel is the remedy, and the project’s experiment runner (`scripts/train_opening.py`) uses one.

10.4 What the gate does not measure

Since 9 October the gate evaluates the *sampled self-play actor* against scripted opponents and a frozen copy of itself. Live turns, however, are decided by search, and the actor only chooses team previews, greedily. A candidate that passes therefore improves a decision system that is not the one deployed for turns. Its effect on live play runs through the preview, and through the data the next candidates are trained on. Section 15 lists this construct-validity gap as the most important open problem in the evaluation design.

10.5 A distribution-free certificate (proposed, not implemented)

A lower confidence bound on the improvement would be a stronger deployment rule than a test.

Theorem 10.3 (Paired Hoeffding lower bound). *Fix both decision systems before evaluation. Draw n evaluation units independently from a declared target distribution, and in unit i run both systems on matching conditions. Let $D_i = X_i^C - X_i^I \in [-1, 1]$ have common mean Δ , with $\bar{D} = n^{-1} \sum_i D_i$. Then, for $\delta \in (0, 1)$,*

$$\mathbb{P}(\Delta \geq \bar{D} - \sqrt{2 \log(1/\delta)/n}) \geq 1 - \delta. \quad (28)$$

Proof. Hoeffding’s inequality for independent variables in intervals of width 2 gives $\mathbb{P}(\bar{D} - \Delta \geq \eta) \leq \exp(-2n\eta^2/(4n)) = \exp(-n\eta^2/2)$ [7]. Set $\eta = \sqrt{2 \log(1/\delta)/n}$ and take complements. $\square$

Example 10.4 (How much evidence a margin is). At $n = 60$ and $\delta = 0.05$ the radius is $\sqrt{2 \log 20/60} \approx 0.316$. The production pass, with $\bar{D} = 8/60 \approx 0.133$, has lower bound -0.18 , so the certificate does not establish improvement. A radius of 0.10 needs $n \geq \lceil 2 \log 20/0.01 \rceil = 600$ independent units. The strategic-v1 final, $\bar{D} = 19/160 \approx 0.119$ with radius 0.193 , is also inconclusive under this bound, although its exact test gave $p = 0.0072$. Hoeffding ignores that most pairs are concordant. A variance-adaptive or exact discordant-pair interval is sharper, and is the right next step.

Dependence *within* a pair is allowed; independence *across* units is required. If both sides of a seed are played, the seed is the unit. If games cluster by opponent team, the team is the unit: Section 14 re-tests the search result at that level. For K fixed candidates, use δ/K per candidate.

Implementation: `paired_gate`, `stage` and `promote_ready` in `ml/promotion.py`; `evaluate` in `ml/worker.py`; preregistered panels in `scripts/train_opening.py`; `docs/checkpoint-validation.md`.

11 Decision-time search

The learned actor’s prior looks one turn ahead against a handful of guessed opponent moves. The search agent replaces that guess with the official simulator. It samples complete hypothetical opposing teams, plays every pairing of our and the opponent’s joint actions for one turn, and solves the resulting matrix game. Figure 10 shows the procedure.

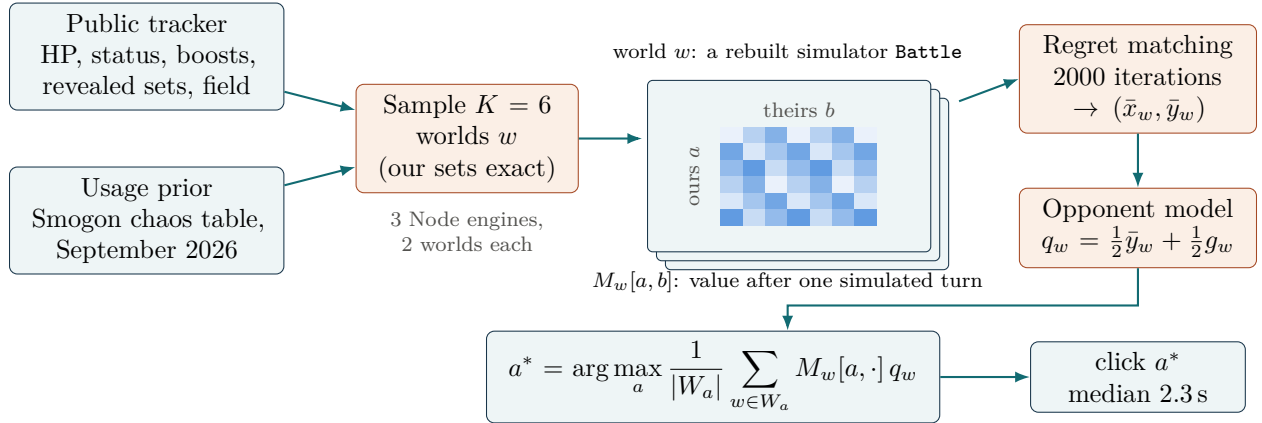


Figure 10: The search agent. Each world fixes complete hypothetical opposing sets and copies all public state. Screening keeps at most 24 of our joint actions and 20 of the opponent’s. The payoff matrix of a world therefore needs at most 480 one-turn simulations, plus up to $4|\mathcal{B}| + 4|\mathcal{A}|$ for screening. Team preview is delegated to the learned actor.

11.1 Public tracker

`search/tracker.py` reads only our own player stream. For each Pokémon it tracks:

- HP fraction (exact HP when the denominator is not 100), status, sleep and toxic counters, boosts and volatiles;
- revealed moves, item (or its loss) and ability;
- Mega state, last move, and the turn it switched in.

For the field it tracks weather, terrain, pseudo-weather and side conditions, each with its start turn. A condition’s remaining duration is estimated as the base duration minus elapsed turns. If that would be non-positive, the extended duration is assumed. Open team sheets are parsed from `|showteam|` lines.

11.2 Sampling hidden sets from usage statistics

The prior is Smogon’s monthly *chaos* usage table for the format (September 2026, all ratings, about 1.6 million battles) [29]. For each usage entry e , which is a species or a Mega forme, the table gives a raw count c_e and normalized frequencies $P_e(\cdot)$ that sum to one over each category. The categories are moves, items, abilities and the 60 most common nature-and-stat-point spreads.

Lemma 11.1 (Forme posterior under conditional independence). *Suppose an opposing Pokémon’s entry is e with prior probability proportional to c_e . Suppose also that, given e , the event “move m is in the moveset” has probability $\pi_e(m)$ independently across moves. Then after observing the revealed set R , $\mathbb{P}(e \mid R) \propto c_e \prod_{m \in R} \pi_e(m)$. If, moreover, $\pi_e(m) = c P_e(m)$ for a constant c shared by all entries (with four move slots, $c \approx 4$), then*

$$\mathbb{P}(e \mid R) \propto c_e \prod_{m \in R} P_e(m). \quad (29)$$

Proof. The first statement is Bayes’ rule with the likelihood factorized by the independence assumption. Under the proportionality assumption, the product becomes $c^{|R|} \prod_{m \in R} P_e(m)$. The factor $c^{|R|}$ is the same for every entry, so it cancels in the normalization. $\square$

The implementation follows Equation (29), with three adjustments:

- unseen moves get a floor of 0.002 on the share scale, so a surprising reveal cannot zero out every entry;
- an observed non-stone item, or a spent Mega, removes the Mega entries;
- an observed Mega keeps only matching Mega entries.

Given e , the unrevealed moves are drawn without replacement in proportion to P_e . Item, ability and spread are each drawn from their own marginals. A Mega entry takes its most common item (assumed to be the stone) and the base forme’s abilities. Unseen previewed Pokémon fill the remaining slots, drawn without replacement in proportion to species usage. When open sheets were exchanged, species, item, ability, moves and nature are copied exactly, and only stat points are sampled.

Caveat. Lemma 11.1 is exact only under its independence assumption. Real movesets have four slots, so move inclusions are negatively dependent. Item, ability and spread are correlated in real sets, and the `Teammates` table is loaded but unused. The Item Clause is not enforced across a sampled team. The result is a tractable proposal distribution, not a calibrated posterior over the opponent’s team.

11.3 Rebuilding a world

Each world becomes a fresh simulator `Battle` built from both packed teams. Public state is then written onto it:

- formes, abilities, items, exact HP for us and $\text{round}(h \cdot \text{maxHP})$ for them, statuses with their counters, and our own PP;

- a whitelist of volatiles, such as Protect’s stall counter, Substitute, Taunt, Encore, Disable and choice lock. The agent does not send last moves, so the engine’s last-move restoration is unused, and the opponent’s PP is never set;
- weather, terrain and side conditions with their estimated durations.

If a side has used its Mega, Mega Evolution is disabled for the whole side. Our side uses exact sets, HP and PP from our request.

11.4 The one-turn game in each world

Our candidates are the Python-legal choices $\mathcal{A}(r)$, pruned in two ways. Damaging moves at our partner are removed, and when a Mega is available only Mega-containing choices are kept. Each surviving choice is rewritten by move *identifier*, because a locked request lists only one move and slot numbers differ in the rebuilt battle. The opponent’s set $\mathcal{B}$ comes from the engine’s own enumeration (Section 4).

Screening. If $|\mathcal{B}| > 20$ or $|\mathcal{A}| > 24$, the engine takes four evenly spaced probe actions $P \subset \mathcal{A}$. It keeps the 20 opponent actions with the lowest mean value against P , which are the best for the opponent. It then keeps the 24 of our actions with the highest mean value against the first four kept replies.

Payoffs. For each remaining pair, the engine restores the world state, gives the cell its own seed, submits both choices, runs exactly one turn and evaluates the result:

$$M_w[a, b] = V(\text{Turn}(B_w; a, b; \xi_{w,a,b})). \quad (30)$$

If the simulator rejects any cell, that cell’s whole row (our action) or column (their action) is removed.

Evaluation function. A finished battle scores ± 30 , or 0 for a tie. Otherwise $V = S(\text{us}) - S(\text{them}) + \text{TR}$, where

$$S = \sum_{\text{alive } p} [1 + 1.2h_p - \text{pen}(p) + \mathbf{1}_{\text{active}} \beta(p)] + 0.10 \cdot \text{Tailwind turns} + 0.06 \sum \text{screen turns}. \quad (31)$$

Status penalties are: paralysis 0.30, freeze 0.50, poison 0.12, toxic 0.25, sleep 0.45 min(1, turns/2), and burn 0.45 if Attack exceeds Special Attack, else 0.12. The active-Pokémon term β rewards offensive, defensive and speed boosts and a Substitute. It penalizes Perish counts, confusion, Taunt, Encore, Yawn and Leech Seed. TR is the Trick Room term: it adds $0.10 \cdot \text{turns} \cdot 2 \text{clip}((\bar{v}_{\text{them}} - \bar{v}_{\text{us}})/40, -1, 1)$, where $\bar{v}$ is each side’s mean Speed, so slower teams gain value under Trick Room.

11.5 Solving the matrix game

The engine runs simultaneous regret matching on the zero-sum game in which we maximize M [6]. With cumulative regrets R_t for us and C_t for the opponent,

$$x_t(i) \propto [R_t(i)]^+, \quad y_t(j) \propto [C_t(j)]^+ \quad (\text{uniform if all} \leq 0), \quad (32)$$

$$R_{t+1}(i) = R_t(i) + (My_t)_i - x_t^\top My_t, \quad C_{t+1}(j) = C_t(j) + x_t^\top My_t - (x_t^\top M)_j, \quad (33)$$

for $T = 2000$ iterations. It returns the average strategies $\bar{x}$ and $\bar{y}$.

Proposition 11.2 (Average regret gives an approximate equilibrium). *Let ε_1 and ε_2 be the two players' average external regrets after T rounds. Then $(\bar{x}, \bar{y})$ is an $(\varepsilon_1 + \varepsilon_2)$ -equilibrium: $\max_i(M\bar{y})_i - \min_j(\bar{x}^\top M)_j \leq \varepsilon_1 + \varepsilon_2$. For regret matching with payoff range Δ , $\varepsilon_1 \leq \Delta\sqrt{|\mathcal{A}|/T}$ and $\varepsilon_2 \leq \Delta\sqrt{|\mathcal{B}|/T}$.*

Proof. Let $\bar{v} = T^{-1} \sum_t x_t^\top M y_t$. By linearity, $T^{-1} \sum_t (M y_t)_i = (M\bar{y})_i$, so our average regret is $\max_i(M\bar{y})_i - \bar{v} \leq \varepsilon_1$. Likewise the opponent's is $\bar{v} - \min_j(\bar{x}^\top M)_j \leq \varepsilon_2$. Adding the two gives the claim. Because $\min_j(\bar{x}^\top M)_j \leq \bar{x}^\top M\bar{y} \leq \max_i(M\bar{y})_i$, neither player can gain more than $\varepsilon_1 + \varepsilon_2$ by deviating. The regret rates are the standard bounds for regret matching [6, 35]. $\square$

Interpretation. With $|\mathcal{A}| = 24$, $|\mathcal{B}| = 20$ and $T = 2000$, the worst-case gap is $\Delta(\sqrt{24} + \sqrt{20})/\sqrt{2000} \approx 0.21\Delta$. When a terminal ± 30 is reachable, Δ can be 60, which makes the bound loose. In practice regret matching usually converges much faster than its worst case. The bound justifies $\bar{y}$ only as an approximate equilibrium, and only of this one-turn surrogate game.

11.6 The opponent model and the decision rule

The agent does *not* play $\bar{x}$. It treats the opponent as a mixture of two types:

$$q_w = \alpha \bar{y}_w + (1 - \alpha) g_w, \quad g_w(b) \propto \exp(-(c_b - \min_{b'} c_{b'})/\tau_o), \quad c_b = \frac{1}{|\mathcal{A}|} \sum_a M_w[a, b], \quad (34)$$

with $\alpha = \frac{1}{2}$ and $\tau_o = 1$. One type plays the equilibrium. The other is a soft best response to our playing uniformly, a level-1 quantal response in the sense of McKelvey and Palfrey [12]. Each world then gives our action values $U_w(a) = (M_w q_w)_a$. Let W_a be the worlds in which a was valid, and W_{ok} the worlds that produced any result. The decision is

$$a^* = \arg \max_a \left[\frac{1}{|W_a|} \sum_{w \in W_a} U_w(a) - 0.5 \mathbf{1}\{|W_a| < |W_{\text{ok}}|\} \right]. \quad (35)$$

Proposition 11.3 (Guarantee and exploitability of a best response). *Let v^* be the value of a world's matrix game. For any opponent mixture y , a best response $a^* \in \arg \max_a (M y)_a$ satisfies $(M y)_{a^*} \geq v^*$. Its worst case, $\min_b M[a^*, b]$, can still be strictly below v^* .*

Proof. By the minimax theorem, $v^* = \min_{y'} \max_a (M y')_a \leq \max_a (M y)_a$. For the second claim take matching pennies, $M = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$, with $v^* = 0$ and $y = (\frac{1}{2}, \frac{1}{2})$. Every pure action is a best response, and each loses 1 to some reply. $\square$

Interpretation. Against the *modelled* opponent the pure choice is never worse than the game's value. Against an opponent who anticipates it, a deterministic best response can be exploited, whereas $\bar{x}$ could not be. The design bets that ladder opponents neither compute this matrix nor exploit its argmax. The level-1 component also assumes they reason one step about our options. These are modelling choices, and the live evidence in Section 14 is too small to test them separately.

11.7 Determinization and strategy fusion

Sampling worlds and solving each with full knowledge is *perfect-information Monte Carlo* (PIMC). It is known to suffer from *strategy fusion*, which means acting as if one could choose differently in worlds one cannot tell apart, and from *non-locality* [4]. Yet PIMC often plays well in practice [11]. Information-set search avoids fusion at a higher cost [2]. Here the turn decision avoids fusion at the root: Equation (35) commits to one a for all worlds. Fusion does enter forced-replacement scoring, which takes $\frac{1}{K} \sum_w \max_a U_w(a)$.

Proposition 11.4 (Forced-replacement scores are optimistic). *For any values $U_w(a)$,*

$$\frac{1}{K} \sum_w \max_a U_w(a) \geq \max_a \frac{1}{K} \sum_w U_w(a),$$

with equality if and only if some single action maximizes U_w in every world.

Proof. For each a' and w , $\max_a U_w(a) \geq U_w(a')$. Averaging over w and then maximizing over a' gives the inequality. If a' maximizes in every world, the two sides coincide. Conversely, if no action does, then for every a' some world has a strict gap, so the average on the left exceeds every right-hand average. $\square$

Interpretation. A replacement whose best follow-up depends on the hidden world is scored as if we will learn the world before moving. That inflates its value, so the rule favours flexible-looking replacements whose flexibility is not real. Within each world both simulated sides also know each other’s sets, even when sheets were not shared, so the opponent model is omniscient about us.

11.8 Monte Carlo error

Each cell uses one stochastic sample of accuracy, damage rolls and critical hits. Each cell has its own seed, so two of our actions are not compared under common random numbers. With $K = 6$ worlds, the averaged difference between two actions has world-sampling noise plus within-cell chance noise.

Proposition 11.5 (Ranking two actions). *Suppose the per-world differences $D_w = U_w(a) - U_w(a')$ are independent, lie in an interval of width 2Δ , and have mean $\gamma > 0$. Then the average over K worlds ranks a' above a with probability at most $\exp(-K\gamma^2/(2\Delta^2))$.*

Proof. This is Hoeffding’s inequality applied to the event $\bar{D} \leq 0$ with deviation γ [7]. $\square$

With $K = 6$, the bound is useful only when the gap γ is a large fraction of Δ . Small value differences are effectively ranked at random. Common random numbers across rows, more seeds per cell, or adaptive allocation of worlds to close decisions would all reduce this noise.

11.9 Preview, Mega and the learned value

Team preview is chosen by the actor, greedily. A value-based preview mode (score 90 plans by the position after preview, averaged over 6 worlds and 3 sampled opposing bring-fours) exists but is untested. Mega Evolution is always taken when available, for us and for the opponent model, so the agent never considers saving it. A small value network can replace V : 14 features per Pokémon plus field and matchup features, a $128 \rightarrow 128 \rightarrow 1$ ReLU MLP, trained by binary cross-entropy on omniscient self-play. It blends in as $(1 - \beta)V + \beta \cdot 8 \tanh(z/2)$, but the deployment uses $\beta = 0$, and the network is neither trained to completion nor gated.

Implementation: `search/agent.py` (worlds, pruning, move-id rewriting, decision rule, forced replacements), `search/engine.cjs` (rebuild, enumeration, screening, payoffs, regret matching, evaluation), `search/priors.py`, `search/tracker.py`, `search/valuefeat.cjs`, `search/train_value.py`; `docs/search-agent.md`.

12 One live turn: browser transport, guards and viewer

12.1 The sequence

Figure 11 follows one turn from the server’s request to the viewer’s animation. The browser runner holds an account lock, so only one controller can drive the account. It reads the official client’s state through `app.rooms[id]`, using the request, `battle.stepQueue`, the choice state and the preview order, and acts only by clicking visible controls. It opens no protocol socket of its own.

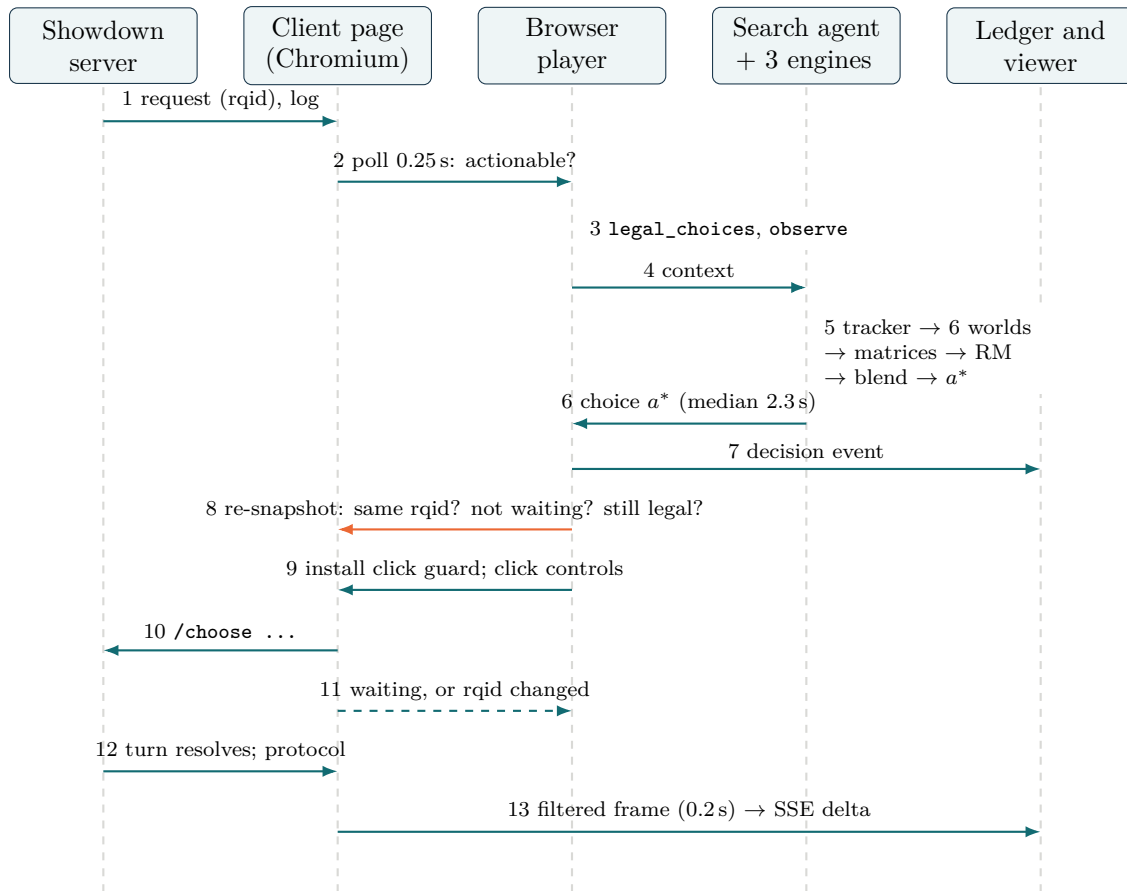


Figure 11: One live turn under search. Step 8 is the freshness check after the decision. Step 9 installs a capture-phase guard that cancels any click once the server’s rqid has changed. Preview turns replace steps 4–6 with a greedy actor decision. In recorded mode, steps 4–6 are `Brain.decide`.

12.2 Freshness guards

Search takes seconds and the client keeps animating, so the request on screen can change while a decision is being computed. Three checks keep a stale decision from being clicked:

1. After deciding, the runner takes a new snapshot. The server rqid must be unchanged, the client must not already be waiting, and the choice must be legal under the new request. Only the rqid is compared, because the client mutates its own copy of the request.
2. A capture-phase listener on the document cancels any click in the room if `request.rqid` differs from the guarded rqid, or if the client is waiting.

3. Before each control the runner re-checks the rqid, and it aborts if the guard has fired.

Proposition 12.1 (Clicks act only on the decided request). *Assume that the server assigns a fresh rqid to each request, and that the page runs JavaScript on a single-threaded event loop. Then every click that reaches the client’s own handler does so while the displayed request has the rqid the decision was computed for, and while the client is not waiting.*

Proof. A click event is dispatched to the capture listener and to the client’s handler within one task of the event loop. A WebSocket message that would replace the request is processed in a separate task, so it cannot run between the guard’s check and the handler. The guard lets the event proceed only if the rqid equals the decided one and the client is not waiting. Since rqids are fresh, equal rqids identify the same request. $\square$

Caveat. The proposition protects individual clicks. A multi-click choice (Mega checkbox, move, target) can be cut off midway when the request changes, leaving a partial choice that the next request supersedes. Server acceptance is confirmed separately, by waiting up to 10s for the client to show waiting or a new rqid. A new `|error|`, or an unchanged rqid with a changed fingerprint, counts as a rejection. At most three submissions are made per rqid, that is, two retries; a third rejection stops the run.

12.3 Recovery

Two recovery paths exist:

- **Recorded browser runs.** A run that restarts mid-game resubmits a saved, unsubmitted proposal only if its stored request equals the current one. If the saved proposal was already submitted, the runner stops rather than guessing, because the server’s acknowledgement is ambiguous.
- **MCP path.** `Brain.resume` restores one compatible pending episode after a transport failure, with the same identity, steps and log-probabilities. It refuses ambiguous fragments or schema changes, and resubmits a saved action for the same rqid without resampling, with up to three attempts per room.

The first edition said that a crash loses the session with no mid-game reconstruction. That is no longer true, within these limits.

Proposition 12.2 (Scope of submission idempotence). *Within one serialized session, a repeated submission for a cached request sends no second choice unless rejection handling removed the cache entry. Repeated finalization returns the cached terminal result. A complete stored episode is never overwritten.*

Proof. The choose path checks its cache key before selecting or sending, and finish returns an existing terminal result first. The store returns early when the stored identifier is already complete. $\square$

This is not an exactly-once guarantee across processes. A send completing on the socket is not a server acknowledgement, and an external submission can still interleave.

12.4 The live viewer

The runner publishes filtered public protocol, with names replaced by “Your agent” and “Opponent”, to an atomically replaced relay file every 0.2s. A small server streams appended lines as server-sent events [34]. It sends a full snapshot on a room change or reconnect, and serves only over the tailnet with HTTPS. The client plays each turn in full. With two or more turns queued it switches to a faster speed. With four or more it jumps to the newest turn. A renderer stuck for 15s is skipped forward. The viewer is for observation only and plays no part in decisions.

Implementation: `ml/browser.py` (snapshot, context, guards, submit, recovery, watch); `scripts/browser_play.py`; `scripts/live_watch.py`; `web/live-watch/`; `ml/live.py` (resume, transport recovery); `docs/browser-play.md`; `docs/live-watch.md`; Playwright [14].

13 Opponent-move scouting and team selection

13.1 The public-move scout

Public logs contain executed events, not private requests or a collecting policy’s probabilities. The repository therefore uses them to train a classifier, the scout, and never fabricates PPO samples from them. `ingest_public` takes a snapshot of each player’s public view at every turn boundary. A later executed move becomes a label for that snapshot only if all of the following hold:

- the actor was already on the field at the boundary;
- the actor is still unambiguous when the move happens;
- the move is in the dex;
- the move is not tagged `[from]` (called or forced).

Chat, authentication, HTML and private request lines are removed, and players are redacted. Incomplete games and orphan samples, whose source battle row is missing, are excluded.

Proposition 13.1 (Turn-boundary non-anticipation). *If the supplied log is chronological, each retained scout vector is a function of the prefix up to its turn boundary and of an actor visible in that prefix. Later moves in the same turn, and later reveals, do not enter it.*

Proof. At a `turn` event the importer builds the context from the prefix up to that boundary. When a later move arrives, it encodes the saved context; the current prefix is used only to reject ambiguous actors. No later prefix is substituted into a saved context. $\square$

The selection of which examples to keep does use later identity checks. The proposition is about feature content, not selection independence.

The scout is an MLP: $384 \rightarrow 96 (\tanh) \rightarrow M$. For the live format $M = 938$ moves and the network has 127,946 parameters. It minimizes

$$\mathcal{L}_{\text{scout}}(\psi) = -\frac{1}{N} \sum_n \log q_\psi(m_n | u_n) \quad (36)$$

with Adam at learning rate 10^{-3} , batches of 128, three epochs and gradient norm 1. Training uses at most 20,000 rows. That cap now binds. The rows used are the 20,000 with the smallest hash identifiers, a fixed pseudo-random tenth of the 192,835 M-C samples. A new sample enters training only if its identifier sorts inside the cap, so the subset behaves like a uniform subsample that refreshes slowly. Battles whose digest is $0 \bmod 5$ form the held-out set, so no battle is split

across partitions. A new scout is promoted only with at least eight held-out samples and no increase in held-out cross-entropy. At inference the softmax is restricted to the moves on a disclosed sheet, or to the moves historically seen for the species. The three most likely moves become hashed state features. Their probabilities are not renormalized, so their sum can be below one.

Proposition 13.2 (Cross-entropy decomposition). *For a true label distribution p and a prediction q positive on p ’s support, $-\sum_m p(m) \log q(m) = H(p) + D_{\text{KL}}(p||q)$. The left side is minimized exactly at $q = p$.*

Proof. Add and subtract $\sum_m p \log p$. Then $D_{\text{KL}} \geq 0$ by Jensen’s inequality, with equality only at $q = p$. $\square$

Caveat. The optimum concerns the distribution of *retained executed* moves, not opponents’ intentions. Inference may also run mid-turn, for example at a forced replacement, whereas training contexts are taken at turn starts, so there is a possible distribution shift. Empirically, the scout lowered held-out cross-entropy, but in a matched battle comparison it changed no outcomes (0 improved, 0 worsened in 200 games), and recorded-state probes show the actor responds only weakly to scout features (docs/offline-ppo-followup.md).

13.2 Team versions as bandit arms

A team arm is an exact ordered list of sets together with its format. Its identifier is

$$v = \text{SHA256}(\text{canonicalJSON}(\{\text{format}, \text{sets}\}))_{1:20},$$

an 80-bit truncated fingerprint. Canonicalization sorts object keys, not the party list. Completed outcomes Y_i become scores $z_i = (Y_i + 1)/2$, and local and ladder evidence are queried separately. The posterior uses a Beta(1, 1) prior:

$$\alpha_v = 1 + \sum_i z_i, \quad \beta_v = 1 + \sum_i (1 - z_i), \quad \mu_v = \frac{\alpha_v}{\alpha_v + \beta_v}, \quad \sigma_v^2 = \frac{\alpha_v \beta_v}{(\alpha_v + \beta_v)^2 (\alpha_v + \beta_v + 1)}. \quad (37)$$

Proposition 13.3 (Conjugacy, and the tie extension). *Suppose binary outcomes are conditionally independent and stationary given p_v . Then w wins and l losses update Beta(α_0, β_0) to Beta($\alpha_0 + w, \beta_0 + l$). Fractional tie scores follow the same algebra under the pseudo-likelihood $\prod_i p_v^{z_i} (1 - p_v)^{1 - z_i}$, which is not a Bernoulli model of a three-outcome game.*

Proof. Multiply the density $\propto p^{\alpha_0 - 1} (1 - p)^{\beta_0 - 1}$ by $p^w (1 - p)^l$. Replacing the integer counts by score sums gives the same exponents, but a Bernoulli variable cannot take the value $\frac{1}{2}$. $\square$

Example 13.4 (A played and an unplayed team). *Seven wins and three losses give Beta(8, 4), $\mu = 2/3$ and $\sigma = \sqrt{32/1872} \approx 0.131$. An unplayed team has $\mu = 1/2$ and $\sigma = \sqrt{1/12} \approx 0.289$.*

With exploration enabled, the planner draws $\tilde{p}_v \sim \text{Beta}(\alpha_v, \beta_v)$ independently and plays $\arg \max_{v \in C} \tilde{p}_v$. This is Thompson sampling [32].

Proposition 13.5 (Probability matching). *Assume independent continuous posteriors over a fixed pool C . Then Thompson sampling selects arm v with probability equal to the posterior probability that v has the largest true success parameter: $\mathbb{P}(\text{select } v) = \mathbb{P}_{\text{post}}(p_v = \max_{u \in C} p_u)$.*

Proof. The sampled vector $(\tilde{p}_u)_{u \in C}$ has the same joint law as the true parameters $(p_u)_{u \in C}$ under the posterior. The event “ v attains the maximum” is therefore equally likely for both vectors. Continuity makes ties have probability zero. $\square$

Interpretation. The first edition stated this with “largest *sampled* parameter”, which is circular: the algorithm selects that arm by definition. The content of the theorem is that sampled and true maxima share one law.

The estimate describes success *as piloted by the policy of the time, against the opponents of the time*. It is not intrinsic team quality, and the posterior is not conditioned on the actor revision. In production the team was pinned (one candidate, exploration off). Thompson selection was therefore inactive in the later campaigns, and every live result in Section 14 uses a single rain team. Variants replace one member with an attributed same-species set and are validated in the simulator. Missing stat spreads are generated, labelled as such, and follow the move categories. A generated Garchomp spread that once boosted the wrong attacking stat was fixed for this reason.

Implementation: `ingest_public`, `public_lines` and `Scout` in `ml/scout.py`; `rank_teams`, `TeamPlanner.plan`, `suggest_variants` and `generated_spreads` in `ml/teams.py`; `fingerprint` in `ml/storage.py`.

14 Evidence

This section collects every quantitative result the project has recorded, recomputes the exact statistics, and states what each one does and does not show. Paired tests use the exact one-sided sign test of Section 10; all p-values below were recomputed and match the validation documents. Unless marked otherwise, sources are the repository’s `docs/*-validation.md`, `docs/opening-policy.md`, `docs/post-campaign-learning.md` and `docs/search-agent.md`.

14.1 Paired candidate evaluations

Figure 12 shows the paired experiments on one scale: the candidate’s win rate minus the incumbent’s, on the same seeds. Different rows use different panels, opponents and incumbents, so their totals are not comparable. Only the direction and the uncertainty of each row are.

Three patterns stand out.

- **Inference-time changes passed; learned weights mostly did not.** The large, independently confirmed gains came from the turn temperature (same weights, $\tau = 0.25$: 95 $\rightarrow$ 129 of 200) and from the strategic prior (125 vs. 106 of 160 on an untouched final). These are changes to how the network’s scores are used, not to its learned residual.
- **Development wins regress.** `matchup-v1` (development $p = 0.0068$, final tie) and `strategic-v4` ($p = 0.0082$, then +1.5 points on fresh seeds) are textbook winner’s curses (Section 10.3). Strata of the v4 confirmation show the tactical-opponent group *regressing*, from 45 to 38 wins of 55.
- **Search is the largest single effect.** The search agent’s paired improvement, +24 points, is larger than any learned candidate’s, with an interval that excludes zero.

14.2 The search agent offline

The opponent in the paired test is the search agent itself, piloting teams that real opponents brought, so the comparison is search against search versus policy against search. Two incumbent-arm games had a degraded opponent (12 fallbacks from a forme lookup error, Appendix C). They were losses in both arms, so they do not change the discordant counts. The search configuration used for these runs was not recorded in the run files, which limits exact reproduction.

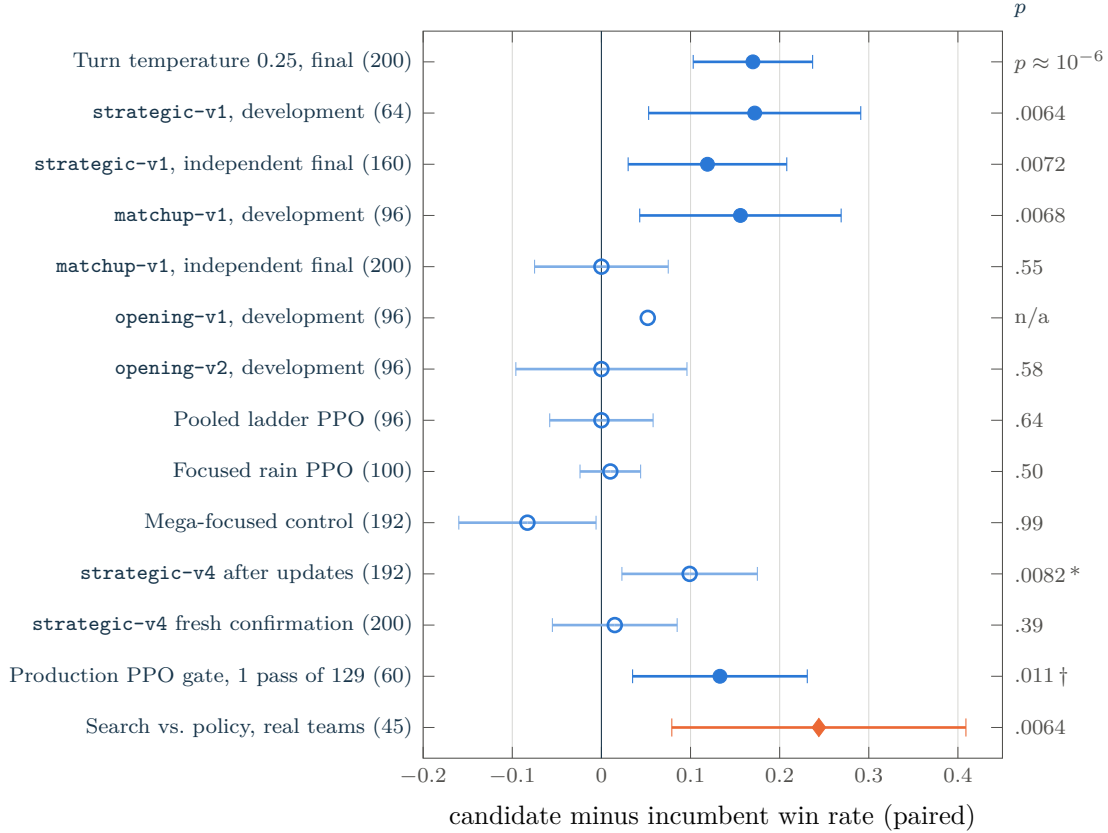


Figure 12: Paired evaluations. Filled markers passed their gate and hollow markers failed. The orange diamond is the search agent, compared with the learned policy on the same 45 seeds. Bars are approximate 95% intervals for a paired difference, $\pm 1.96\sqrt{[(g + \ell)/n - ((g - \ell)/n)^2]/n}$. The **opening-v1** panel did not report its discordant counts. * Met $p \leq 0.05$ but missed the then-required 10-point minimum by one win. † One pass among 129 gates; see Proposition 10.2.

Test	Result
Rain mirror, head to head, 40 games, sides alternated	Search 26–14 (65%); exact binomial one-sided $p = 0.040$.
Real teams, paired: same 45 seeds, sides and opponent teams; the opponent is search-piloted	Policy 19/45 (42%), search 30/45 (67%); 14 gains, 3 losses; McNemar one-sided $p = 0.0064$ (two-sided 0.013).
Same, clustered by opponent team (23 teams, each from both sides, one once)	13 teams favour search, 2 the policy, 8 tie; sign test $p = 0.0037$; exact sign-flip permutation $p = 0.012$.
Calibration: policy vs. search-piloted real teams	69/140 (49.3%), Wilson 95% [41.1%, 57.5%]; close to the live 41%.
Synthetic usage-sampled teams	Policy wins 9–1: <i>not</i> a valid proxy for the ladder.

Table 8: Offline search evidence, with open team sheets off. The run ledgers were recomputed for this report. `docs/search-agent.md` describes the paired panel as “45 real human teams”; it contains 45 games over 23 distinct teams. The team-level re-test shows that the effect survives this clustering.

14.3 Why raw ladder win rate is not a strength measure

Showdown’s ladder uses an Elo rating with a floor of 1000 and a rating-dependent K [18]. A player of rating R facing R_o has expected score $E = 1/(1 + 10^{(R_o - R)/400})$ [3]. A win adds $K_w(1 - E)$ and a loss subtracts $K_\ell E$. Above 1300, $K_w = K_\ell = 40$; from 1100 to 1299 both are 50. At 1000 the winner gains with $K_w = 80$ and the loser loses with $K_\ell = 20$, and these interpolate linearly up to 1100.

Proposition 14.1 (Stationary win rate under asymmetric K). *Suppose a player keeps a constant true win probability p against opponents with expected score E , so that its expected rating drift is zero. Then*

$$p = \frac{K_\ell E}{K_w(1 - E) + K_\ell E}.$$

For evenly matched opponents ($E = \frac{1}{2}$), this is $K_\ell/(K_w + K_\ell)$: 0.20 just above the floor, about 0.35 near 1050, and 0.50 from 1100 upward. The formula assumes no clamp, $R - K_\ell E \geq 1000$. At the floor itself a loss costs nothing, so the stationary p is lower still.

Proof. The expected change is $pK_w(1 - E) - (1 - p)K_\ell E$. Setting it to zero and solving for p gives the formula. Substituting $E = \frac{1}{2}$ gives the special case. $\square$

Interpretation. A player who is stable in the 1000–1100 band wins at most 50%, and as little as 20% or less near the floor, simply because of the asymmetric K and the clamp. The 847-game campaign’s 41.1% is consistent with a policy that hovered in that band. A live goal of “50% win rate” is therefore partly a statement about the rating band. A steadily rising rating above 1100 is the more informative signal. GXE or a fixed-rating evaluation pool would be better still.

14.4 Live record

Games	Decision system	Record	Win rate
1–100	Legacy actor; temperature promoted before game 69	33–67	33%
101–375	Turn-temperature actor (overnight campaign)	113–162	41%
376–500	Same actor; corrected rain team	50–75	40%
433–532	Last 100 before strategic-v1	36–64	36%
533–632	strategic-v1 , first 100	49–51	49%
1–847	All three generations	348–499	41% [38, 44]
41 games, 9 Oct	Search turns; greedy actor preview	27–14	66% [51, 78]

Table 9: Live ladder records with Wilson 95% intervals. These periods are not randomized comparisons. Team, rating band, opponent pool and transport all changed between them. The search row counts each battle room once (as of 20:07 UTC). It includes two losses in which no search decision was submitted, because of transport or team-validation incidents. Two battles abandoned to the timer during earlier transport crashes are not in this ledger; counting them gives 27–16.

Testing the live search record against Elo. Over the 37 search games whose logs give both players’ pre-game ratings, Elo predicts $\sum_i E_i = 18.4$ wins, with variance $\sum_i E_i(1 - E_i) = 8.97$. We observed 23 wins, $z = 1.53$. The exact Poisson–binomial upper tail is $\mathbb{P}(X \geq 23) = 0.086$. Taken alone, the live sample is suggestive but not significant at 0.05. With the rating climb and the offline paired result, it is consistent with a real gain. It is *not* an independent confirmation of the offline

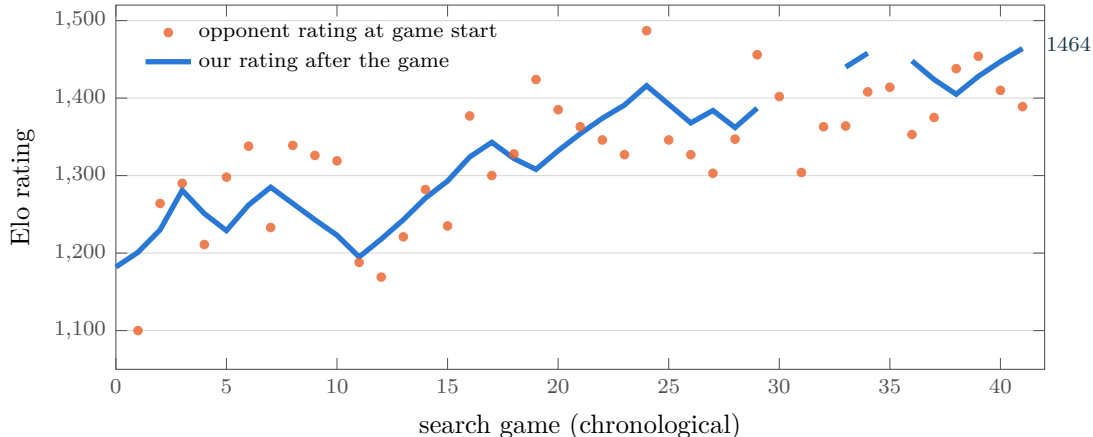


Figure 13: Live rating under the search agent on 9 October 2026. Gaps are games whose terminal log lacked our rating line. The climb from 1182 to 1464 took place in the symmetric- K region, where Proposition 14.1 predicts no drift for a player of stable strength.

effect size. A pre-specified live sample of a few hundred games, or rating trajectories compared over equal game counts, would settle it.

14.5 Offline PPO hyperparameter matrix

Seven settings (learning rate, entropy, collection size, minibatch, epochs and entropy decay) were trained with three seeds and five updates each. Each was evaluated against random and tactical opponents, with a sign test, a conditional bootstrap, and Holm correction over 14 development and 12 final-versus-control hypotheses [8]. No arm passed after correction. Two features of the design limit what it shows. The initial heuristic policy was already strong against random play, and arms with larger collections also took far fewer optimizer steps, which confounds data size with optimization exposure. The production hyperparameters were retained (docs/offline-ppo-followup.md).

14.6 Negative results and defects found along the way

An honest record includes what went wrong. The following were found and fixed, or are still open, during 7–9 October:

- **Label errors.** Five early wins were stored as losses after an identity change (corrected). A coloured public HP string parsed as full HP, which is still live in the deployed profile’s state encoder (Section 3). Weather Ball was encoded as a 50-power Normal move in rain. A generated Garchomp spread boosted the wrong attacking stat.
- **Design errors.** Simulator seeds were truncated to 16 bits. The opponent schedule was confounded with team index. Fourteen orphaned scout labels were found. A score cap flattened 288 of 360 preview choices. Postgame feedback was silently dropped by the immutable-write rule.
- **Behavioural limits.** Preview entropy was 97.9% of uniform. The preferred four changed in only 4 of 100 cases when the opponent’s six changed. The pressure curriculum never exercised its intended setup move.

- **Search defects** (Appendix C): a both-Mega enumeration drop, a forme lookup failure, unseeded sleep and confusion durations, and no common random numbers.

15 Limitations and research directions

The system is a working, auditable player with one strong offline result and a long list of honest failures. Its main open problems follow from the analysis above, in rough order of importance.

1. **Evaluate the system that is deployed.** The promotion gate measures the sampled self-play actor. Live turns are chosen by search. A candidate actor should be evaluated as a *preview* policy under search turns. Search changes (worlds, opponent blend, evaluation) need a gate of their own, with the opponent team as the unit (Sections 10 and 14).
2. **Control multiplicity.** The pipeline runs about a hundred gates a day. A fixed $\alpha = 0.05$ per gate admits several false promotions per hundred null candidates. Options are an alpha-spending budget, a Bonferroni or Holm level per day, or a mandatory independent final panel, which the experiment runner already uses (Proposition 10.2).
3. **Measure strength on a fixed scale.** Live win rate depends on the rating band (Proposition 14.1). Rating trajectories over pre-specified game counts, or GXE, should replace raw win rate as the live target. A frozen pool of real opponent teams piloted by fixed policies should replace synthetic teams as the offline target.
4. **Fix the search defects and reduce its noise.** The defects are the both-Mega enumeration drop, Mega forcing, forme lookups and unseeded durations. Noise can be cut with common random numbers across rows, more than one seed per cell, and more worlds for close decisions (Proposition 11.5). The optimistic replacement scoring could be replaced by a committed choice across worlds (Proposition 11.4).
5. **Better beliefs.** The usage prior ignores correlations between moves, items and spreads, as well as teammates. A joint set model, or a posterior updated from damage observations (a revealed roll constrains the stat spread), would sharpen the worlds. Information-set search would remove fusion at the cost of compute [2].
6. **State sufficiency.** Fix the legacy HP parser in the deployed profile, distinguish unknown from observed values, and test longer histories or recurrence. An exact simulator does not help if the encoder drops a mechanic.
7. **Make learning matter.** Inference-time changes (temperature, prior, search) produced every confirmed gain, while gradient updates produced almost none. A learned value function for search, trained on self-play with the existing value-net pipeline and gated like any candidate, is the clearest route by which learning could compound with search, in the spirit of Angliss et al. [1], Grigsby et al. [5] and Karten et al. [10].

The system has no theorem of equilibrium convergence, optimal team construction, belief recovery or monotone improvement. Its rigour lies in stating each implemented contract and each proof assumption explicitly, so that every new empirical claim can be tested against a defined system.

A Notation

Symbol	Meaning
s_t, h_t	Latent simulator state; information available before decision t .
$r, \mathcal{A}(r)$	A Showdown request and its request-visible candidate choices.
x_t, z_a	State vector in $\mathbb{R}^{384}$; action vector in $\mathbb{R}^{192}$.
$q_0, f_\theta, \ell_\theta$	Prior (last action coordinate), learned residual, full logit.
τ	Policy temperature; 0.25 for turns and previews in the deployed checkpoint.
π_θ, V_θ	Tempered masked policy; critic.
$s(a), \kappa, g$	Strategic turn score, pessimism weight, squash $4s/(4 + s)$.
Y, T, G_t	Terminal outcome, recorded decision count, discounted return.
γ, λ	Discount 0.99 and GAE parameter 0.95.
$\hat{A}_t, \tilde{A}_t, \hat{R}_t$	GAE estimate, standardized advantage, critic target.
$\rho_t, \epsilon, \omega_t$	Importance ratio, clip width 0.2, phase weight.
$\bar{D}_{\text{KL}}$	Per-epoch estimate $(\rho - 1) - \log \rho$; stop threshold 0.03.
W, d	Admitted simulator workers; GPU duty fraction.
g, ℓ, d, p	Paired gains, losses, discordant total, exact sign-test p-value (Section 10; d also denotes duty in Section 9).
n, m	Games per policy and promotion margin.
w, K, M_w	Sampled world, world count (6 live), per-world payoff matrix.
$\bar{x}_w, \bar{y}_w, q_w$	Regret-matching averages; blended opponent model.
$U_w(a), a^*$	Value of our action in world w ; chosen action.
E, K_w, K_ℓ	Elo expected score; winner and loser K factors.
ψ, q_ψ	Scout parameters and executed-move distribution.
$v, \alpha_v, \beta_v, \mu_v, \sigma_v$	Team version and its Beta posterior summaries.

B Corrections to the first edition

First edition	This edition
Snapshot 151393d	A private-history hash absent from the public repository. This edition cites public 0c00ec2.
The actor chooses every live decision	Search chooses turns; the actor chooses previews greedily (Section 11).
Policy softmax(ℓ); example (0.594, 0.343, 0.063)	The policy is softmax(ℓ/τ) with $\tau = 0.25$. The deployed probabilities for the same logits are (0.900, 0.100, 0.0001) (Example 6.2).
Prior: power $\times$ accuracy $\times$ STAB $\times$ type, zero at preview	This is the legacy prior. The deployed prior is the squashed scenario score, and it is nonzero at preview (Section 5.3).
Preview ordering always matters	Under the strategic profiles the bench is canonicalized: 360 candidates form 180 plans (Proposition 4.2).
“Not a hypothesis test”; gate max(1, $\lfloor nm + 0.999 \rfloor$)	The gate is $\lceil nm \rceil$ plus an exact paired sign test at $p \leq 0.05$ (Equation (27)).
The worker promotes, with a 30-minute window	The worker only stages. Promotion happens before matchmaking, with no pending ladder recording, a matching parent, hash and source generation.
Auxiliary inputs are not frozen	Evaluation freezes content-addressed input snapshots.
No mid-game reconstruction after a crash	Pending-episode resume and same-rqid resubmission exist (Section 12).
Loss: clipped PPO + value + entropy	Now with phase weights, configurable coefficients, collecting-likelihood exclusion, quality exclusions and an epoch-level KL stop (Section 8).

First edition	This edition
Consumed “once produced and evaluated”	Consumed when the candidate is saved, before evaluation.
Opponents: heuristic plus some random; optional self	The <code>ladder-v1</code> cycle (2/8 self, 3/8 tactical, 2/8 heuristic, 1/8 random); 8% open sheets; 64-bit seeds.
Resource rule: load, memory, disk	Adds memory pressure (critical only), the swap-out rate, the storage budget, and the split between evaluator and collector.
Thompson: “probability that v has the largest <i>sampled</i> parameter”	Circular; corrected to the largest <i>true</i> parameter (Proposition 13.5).
Hashing into d coordinates	Into $d - 16$ hashed buckets; 16 are dense (Proposition 5.1).
Training mask uses a finite minimum	Divided by $\tau < 1$, the minimum overflows to $-\infty$; this is harmless (Section 6).
“Not a tree search or equilibrium solver”	True of the network. False of the deployed system, which solves per-world matrix games.
Evidence: two early reports	Section 14: more than thirty gates, the search evaluation and the live record.

C Implementation defects found during this review

These are reported for transparency. None was changed by this documentation update.

1. **Both-Mega enumeration drop** (`search/engine.cjs`, `sideChoices`). When both actives on a side can Mega Evolve, every move option carries `mega`. Each move-and-move pair then has two Mega events and is filtered out. Pairs that include a switch survive, so the list is not empty and the repair branch never runs; `megaFix` is a no-op. The side is left with only switch-containing actions. This affects opponent worlds in which both sampled actives hold stones, and our own forced-replacement look-ahead before our Mega is spent. The fix is to emit each move with and without `mega` when `canMegaEvo`, and let the one-Mega filter choose.
2. **Mega always forced**. Delaying Mega Evolution is never considered, for us or for the opponent model.
3. **Forme lookup**. `_our_set` fails for formes whose Mega base name differs (for example Floette-Eternal and Floette-Mega). An arena opponent fell back 12 times for this reason.
4. **Unseeded durations and independent cell seeds**. Rebuilt sleep and confusion durations use `Math.random`, and each matrix cell has its own seed, so rows are not compared under common random numbers.
5. **Legacy HP parsing** in the deployed state encoder (Section 3).
6. **Floating-point margin**. $\lceil nm \rceil$ can exceed the exact ceiling for some (n, m) (Section 10).
7. **Dead constant**. The burn penalty in the evaluation table is overridden by the attack-based rule.
8. **Incomplete world state**. Last moves are never sent to the engine, so its last-move restoration (and anything that depends on it, such as Encore targets) is unused. The opponent’s PP is never set.
9. **Coarse invalidation**. One rejected matrix cell removes a whole row or column of the world’s game.

D Source-to-claim map

Source	Claim or object
<code>ps_mcp_server.py</code> , <code>ps_client.py</code> , <code>harness.py</code> <code>battle_state.py</code> <code>ml/features.py</code>	41 MCP tools; login and transport; team CRUD; legacy loop. <code>observe</code> ; <code>legal_choices</code> ; joint predicates; HP parsers. Dimensions 384/192; signed hashing; profiles; bench canonicalization; prior placement and squash.
<code>ml/strategy.py</code> , <code>ml/tactics.py</code> , <code>ml/opening.py</code> <code>ml/model.py</code>	Scenario hypotheses; deterministic resolution; risk aggregate; damage; preview score. 97,826-parameter network; temperature; GAE; likelihood validation; PPO, BC and KL stop; duty cycle.
<code>ml/brain.py</code> , <code>ml/recording.py</code> , <code>ml/postgame.py</code> <code>ml/data_quality.py</code> <code>ml/storage.py</code> , <code>ml/input_snapshot.py</code> , <code>ml/data_budget.py</code> <code>ml/pipeline.py</code> , <code>ml/worker.py</code>	Decisions, steps, snapshots, terminal records, resume. Trajectory exclusion rules. Schema, fingerprints, collected checkpoints, frozen inputs, budget.
<code>ml/resources.py</code> , <code>ml/batching.py</code> , <code>ml/reload.py</code> <code>ml/promotion.py</code> <code>ml/scout.py</code> , <code>ml/teams.py</code> , <code>ml/research.py</code>	Supervisor; lanes; collection curriculum; learner; evaluator. Admission and guards; MPS backend; tensor cache; source generation.
<code>search/agent.py</code> , <code>search/engine.cjs</code> , <code>search/priors.py</code> , <code>search/tracker.py</code> <code>ml/browser.py</code> , <code>scripts/browser_play.py</code> , <code>scripts/live_watch.py</code> <code>simulator.cjs</code> , <code>ml/simulator.py</code> <code>experiments/</code> , <code>notebooks/</code>	Paired gate; sign test; staging; promotion. Scout; Beta/Thompson team choice; research prior. Worlds; enumeration; screening; regret matching; blend; decision; evaluation. Browser turn; guards; recovery; viewer. Official simulation and validation; player streams; local opponents. Offline PPO matrix and analysis.

E Reproduction and build

Build from `docs/paper/` with `tectonic main.tex` (or `latexmk -pdf main.tex`). The figures are TikZ and pgfplots, with no external images. To reproduce an experiment, record at least:

1. repository and simulator revisions, format, dependency versions, feature profile, temperatures, device and seeds;
2. exact teams with their order and generated-spread flags, learner side, opponent policy and revision;
3. frozen input snapshots, collection mode and terminal, unfinished and rejected counts;

4. for search: worlds, engines, screening sizes, the usage month and cutoff, α , τ_o , iterations and the seed;
5. the evaluation unit (game, seed or opponent team), the panel’s independence from development, and the multiplicity plan.

Use an isolated data root for experiments, because the ordinary root is live learning state. The offline test suite checks the implemented contracts (enumeration, masking, collection modes, likelihoods, terminal recording, reloads, scout non-anticipation, promotion predicates). Passing it is not a measurement of playing strength.

References

- [1] Cameron L. Angliss, Jiaxun Cui, Jiaheng Hu, Arrasy Rahman, and Peter Stone. VGC-Bench: Towards mastering diverse team strategies in competitive Pokémon. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS, 2026. URL <https://www.cs.utexas.edu/~pstone/Papers/bib2html-links/angliss2026vgc.pdf>.
- [2] Peter I. Cowling, Edward J. Powley, and Daniel Whitehouse. Information set Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):120–143, 2012. doi: 10.1109/TCIAIG.2012.2200894. URL <https://eprints.whiterose.ac.uk/75048/>.
- [3] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Publishing, New York, 1978.
- [4] Ian Frank and David Basin. Search in games with incomplete information: A case study using bridge card play. *Artificial Intelligence*, 100(1–2):87–123, 1998. doi: 10.1016/S0004-3702(97)00082-9. URL <https://www.dai.ed.ac.uk/daidb/papers/documents/rp780.html>.
- [5] Jake Grigsby, Yuqi Xie, Justin Sasek, Steven Zheng, and Yuke Zhu. Human-level competitive Pokémon via scalable offline reinforcement learning with transformers. *Reinforcement Learning Journal*, 6:2685–2719, 2025. URL <https://arxiv.org/abs/2504.04395>.
- [6] Sergiu Hart and Andreu Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica*, 68(5):1127–1150, 2000. doi: 10.1111/1468-0262.00153. URL <https://doi.org/10.1111/1468-0262.00153>.
- [7] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830. URL <https://doi.org/10.1080/01621459.1963.10500830>.
- [8] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.jstor.org/stable/4615733>.
- [9] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X. URL <https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf>.
- [10] Seth Karten, Andy Luu Nguyen, and Chi Jin. Pokéchamp: an expert-level minimax language agent. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, 2025. URL <https://proceedings.mlr.press/v267/karten25a.html>.
- [11] Jeffrey Long, Nathan R. Sturtevant, Michael Buro, and Timothy Furtak. Understanding the success of perfect information Monte Carlo sampling in game tree search. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, pages 134–140, 2010. URL <https://ojs.aaai.org/index.php/AAAI/article/view/7562>.

- [12] Richard D. McKelvey and Thomas R. Palfrey. Quantal response equilibria for normal form games. *Games and Economic Behavior*, 10(1):6–38, 1995. doi: 10.1006/game.1995.1023. URL <https://resolver.caltech.edu/CaltechAUTHORS:20171128-164515991>.
- [13] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996. URL <https://link.springer.com/article/10.1007/BF02295996>.
- [14] Microsoft. Playwright, n.d. URL <https://playwright.dev/>. Browser automation library; accessed 2026-10-09.
- [15] Model Context Protocol contributors. Model context protocol specification: Tools, 2025. URL <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>. Specification version 2025-06-18; accessed 2026-10-07.
- [16] Model Context Protocol contributors. Model context protocol specification: Transports, 2025. URL <https://modelcontextprotocol.io/specification/2025-06-18/basic/transports>. Specification version 2025-06-18; accessed 2026-10-07.
- [17] OpenClaw contributors. Manage saved mcp servers, n.d. URL <https://docs.openclaw.ai/cli/mcp/registry>. Accessed 2026-10-07.
- [18] Pokémon Showdown. How the ladder works, n.d. URL <https://pokemonshowdown.com/pages/ladderhelp>. Accessed 2026-10-09.
- [19] Pokémon Showdown contributors. Battleground and player stream implementation, n.d.. URL <https://github.com/smogon/pokemon-showdown/blob/c046106cbe075931b1ff8d8b800ff5be47a85f96/sim/battle-stream.ts>. Pinned official simulator source; inspected through repository dependency.
- [20] Pokémon Showdown contributors. Pokémon showdown client–server protocol, n.d.. URL <https://github.com/smogon/pokemon-showdown/blob/c046106cbe075931b1ff8d8b800ff5be47a85f96/PROTOCOL.md>. Pinned simulator source; inspected through repository dependency.
- [21] Pokémon Showdown contributors. Pokémon showdown: Pinned simulator and format source, n.d.. URL <https://github.com/smogon/pokemon-showdown/tree/c046106cbe075931b1ff8d8b800ff5be47a85f96>. Commit c046106cbe075931b1ff8d8b800ff5be47a85f96; project dependency pin.
- [22] Pokémon Showdown contributors. Pokémon showdown simulator protocol, n.d.. URL <https://github.com/smogon/pokemon-showdown/blob/master/sim/SIM-PROTOCOL.md>. Accessed 2026-10-07.
- [23] PyTorch contributors. torch.mps.set_per_process_memory_fraction, n.d. URL https://pytorch.org/docs/stable/generated/torch.mps.set_per_process_memory_fraction. Accessed 2026-10-09.
- [24] John Schulman. Approximating KL divergence, 2020. URL <http://joschu.net/blog/kl-approx.html>. Blog post, 7 March 2020.
- [25] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, 2015. URL <https://proceedings.mlr.press/v37/schulman15.html>.
- [26] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016. URL <https://arxiv.org/abs/1506.02438>.

- [27] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347. URL <https://arxiv.org/abs/1707.06347>.
- [28] Richard D. Smallwood and Edward J. Sondik. The optimal control of partially observable markov processes over a finite horizon. *Operations Research*, 21(5):1071–1088, 1973. doi: 10.1287/opre.21.5.1071. URL <https://pubsonline.informs.org/doi/10.1287/opre.21.5.1071>.
- [29] Smogon University. Pokémon showdown usage statistics (chaos JSON), september 2026, 2026. URL <https://www.smogon.com/stats/>. Format gen9championsvgc2026regmc, cutoff 0; 1,631,943 battles.
- [30] Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html.
- [31] The Pokémon Company. Regulation set m-c (updated on september 9), 2026. URL <https://champions-news.pokemon-home.com/en/page/816.html>. Accessed 2026-10-07.
- [32] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933. doi: 10.1093/biomet/25.3-4.285. URL <https://academic.oup.com/biomet/article-abstract/25/3-4/285/200862>.
- [33] Kilian Weinberger, Anirban Dasgupta, Josh Attenberg, John Langford, and Alex Smola. Feature hashing for large scale multitask learning. *arXiv preprint arXiv:0902.2206*, 2009. URL <https://arxiv.org/abs/0902.2206>. Version 5 revised 2010.
- [34] WHATWG. HTML living standard: Server-sent events, n.d. URL <https://html.spec.whatwg.org/multipage/server-sent-events.html>. Accessed 2026-10-09.
- [35] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, volume 20, 2007. URL <https://proceedings.neurips.cc/paper/2007/hash/08d98638c6fcd194a4b1e6992063e944-Abstract.html>.